B.Tech. BCSE497J - Project-I

MEMORA: AI-POWERED EVENT MEDIA DISCOVERY & REEL SYNTHESIS PLATFORM

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering

by

23BKT0053 SAI VARDHAN REDDY
23BCB0043 SRI RITHVIK

Under the Supervision of

Dr. Kaspar S
Associate Professor / Senior Faculty

School of Computer Science and Engineering (SCOPE)



VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

October 2026

DECLARATION

We hereby declare that the project entitled **Memora: Ai-Powered Event Media Discovery & Reel Synthesis Platform**, submitted by us to Vellore Institute of Technology (VIT) for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of bonafide work carried out by us under the supervision of Dr. Kaspar S, Associate Professor, School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Vellore.

We further declare that the work reported in this project has not been submitted, and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place : Vellore

Date : 27-10-2026

Sai Vardhan Reddy	Sri Rithvik

CERTIFICATE

This is to certify that the project entitled **MEMORA: AI-POWERED EVENT MEDIA DISCOVERY & REEL SYNTHESIS PLATFORM** submitted by Sai Vardhan Reddy (23BKT0053) and Sri Rithvik (23BCB0043), School of Computer Science and Engineering, VIT, for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of bonafide work carried out by them under my supervision during Fall Semester 2024–2025, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The project fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place : Vellore

Date : 27-10-2026

Signature of the Guide

Dr. Kaspar S
Associate Professor, SCOPE
Vellore Institute of Technology

Examiner(s):

1. _____

2. _____

Head of the Department:

Dr. Jaisankar N
Dean, School of Computer Science and Engineering
Vellore Institute of Technology

ACKNOWLEDGEMENTS

We sincerely thank the management of Vellore Institute of Technology (VIT) for giving us the opportunity, and the computing resources, to take on a project of this scale. The institute's commitment to a research-oriented learning environment has supported us throughout our studies, and its high-performance computing facilities and digital library allowed us to implement and benchmark current deep learning models properly.

We are grateful to Dr. Jaisankar N, Dean of the School of Computer Science and Engineering (SCOPE), for his administrative support and encouragement. The high standards he sets for technical work, and the culture of innovation and critical thinking he has built within the school, gave us a strong foundation for undergraduate research.

Our deepest thanks go to our project supervisor, Dr. Kaspar S, Associate Professor, School of Computer Science and Engineering. His guidance, technical insight and patient feedback at every milestone were invaluable. His understanding of computer vision pipelines, the finer points of biometric verification and asynchronous distributed systems helped us get past several serious performance problems, and his encouragement led us to try combining multi-scale tiling, FAISS vector search and procedural audio synthesis.

We also thank the faculty and technical staff of the School of Computer Science and Engineering for their cooperation throughout the project. Finally, we thank our parents and friends for their constant support and patience during the ideation, development and testing phases.

Sai Vardhan Reddy (23BKT0053) Sri Rithvik (23BCB0043)

TABLE OF CONTENTS

Sl. No.	Contents	Page No.
	Abstract	xi
1	**INTRODUCTION**	**1**
1.1	Background	1
1.2	Motivations	1
1.3	Scope of the Project	2
2	**PROJECT DESCRIPTION AND GOALS**	**3**
2.1	Literature Review	3
2.1.1	Multi-Task Face Detection and Feature Alignment	3
2.1.2	Efficient Edge Face Localization and Tiling	4
2.1.3	Hyperspherical Loss Formulations and Embeddings	4
2.1.4	Deep Metric Learning and Illumination Invariance	4
2.1.5	High-Dimensional Approximate Nearest-Neighbor Vector Indexing	4
2.2	Research Gap	5
2.3	Objectives	6
2.4	Problem Statement	6
2.5	Project Plan	7
3	**TECHNICAL SPECIFICATION**	**8**
3.1	Requirements	8
3.1.1	Functional	8
3.1.2	Non-Functional	8
3.2	Feasibility Study	9
3.2.1	Technical Feasibility	9
3.2.2	Economic Feasibility	9
3.2.3	Social Feasibility	9
3.3	System Specification	10
3.3.1	Hardware Specification	10
3.3.2	Software Specification	10
4	**DESIGN APPROACH AND DETAILS**	**11**
4.1	System Architecture	11
4.2	Design	13
4.2.1	Data Flow Diagram	13
4.2.2	Use Case Diagram	14
4.2.3	Class Diagram	14
4.2.4	Sequence Diagram	15
4.2.5	Component and Interface Architecture	16

Sl. No.	Contents	Page No.
5	**METHODOLOGY AND TESTING**	**17**
5.1	Module Description	17
5.1.1	Ingestion and Tiling Engine	17
5.1.2	Preprocessing and CLAHE Normalization	18
5.1.3	SCRFD Deep Face Detection	18
5.1.4	ArcFace Embedding Extraction	18
5.1.5	FAISS Vector Similarity & Centroid Clustering	19
5.1.6	Automated Reel Synthesis & Procedural Audio	20
5.2	Experimental Methodology	20
5.3	Testing	21
6	**PROJECT DEMONSTRATION**	**22**
6.1	Web Dashboard & Organizer Management	22
6.1.1	Venue Signage QR Distribution & Worker Telemetry Monitor	23
6.2	Zero-Install Guest Onboarding & Biometric Scanner	24
6.2.1	Live Biometric Kiosk Scanner Station	24
6.3	Automated Photo Matching & Instant Personal Gallery	25
6.3.1	High-Throughput Batch Zip Archive Streaming Exporter	26
6.4	Automated Memory Reel Synthesis & Beat Pacing Lab	27
6.4.1	Real-Time Procedural Web Audio Synthesizer & Dynamic Filter Lab	28
6.5	Cross-Platform Mobile Deployment (Oppo Reno 11 5G)	29
7	**RESULT AND DISCUSSION**	**30**
7.1	Experimental Results	30
7.1.1	RQ1: Multi-Scale Tiling Face Detection Recall	30
7.1.2	RQ2: ArcFace Angular Margin Optimization	31
7.1.3	RQ3: CPU vs GPU Latency & Throughput Trade-offs	31
7.1.4	RQ4: FAISS Vector Scalability vs Brute-Force	32
7.1.5	RQ5: Multi-Angle Centroid Verification Robustness	32
7.1.6	RQ6: Dynamic Beat-Synchronization Precision	33
7.1.7	RQ7: Celery/Redis Fault Tolerance & Thread Fallback	33
7.2	Discussion	34
7.3	Cost Analysis	35
8	**CONCLUSION**	**37**
8.1	Summary of Contributions	37
8.2	Societal and Ethical Impact	37
8.3	Limitations and Future Research Directions	37
9	**REFERENCES**	**39**
	APPENDIX A – SAMPLE CODE IMPLEMENTATION	**43**

LIST OF FIGURES

Figure No.	Title	Page
2.1	Project Timeline and Engineering Work Breakdown (Gantt Chart)	7
4.1	System Architecture of Memora 4-Tier Platform	11
4.2	Level-0 Context Data Flow Diagram (DFD)	12
4.3	Level-1 Functional Decomposition Data Flow Diagram (DFD)	12
4.4	Level-2 Media Ingestion and Face Verification Pipeline (DFD)	12
4.5	Use Case Diagram Across Platform Stakeholders and Actors	13
4.6	Class Diagram of Core Domain Entities and Biometric Models	14
4.7	Sequence Diagram of Guest Biometric Onboarding and Centroid Registration	14
4.8	Sequence Diagram of Batch Media Upload and Asynchronous AI Inference	15
4.9	Sequence Diagram of Automated Beat-Synchronized Reel Synthesis	15
5.1	ArcFace Feature Space and Angular Decision Boundary	18
5.2	Dynamic Beat-Synchronized Reel Timeline & Transition Grid	19
6.1	Dashboard with System Status, Event Management, and Live Ingestion Feed	21
6.2	Venue Signage QR Generator and Live Ingestion Monitor	22
6.3	Guest Onboarding Interface with Multi-Angle Camera Alignment	23
6.4	Live Biometric Kiosk Scanner Modal with Instant Recognition	24
6.5	Instant Personalized Photo Gallery Feed with Confidence Indicators	25
6.6	Batch Download and Streaming Zip Archive Generation Interface	26
6.7	Creative Memory Reel Studio with 128 BPM Beat Grid and Pacing Controls	27
6.8	Procedural Web Audio Synthesizer Controls and Dynamic Color Grading Lab	28
6.9	Capacitor Mobile Native Application Verified on Oppo Reno 11 5G	29
7.1	Receiver Operating Characteristic (ROC) Verification Curve	31
7.2	Vector Query Latency Scaling with FAISS Index Size	32

LIST OF TABLES

Table No.	Title	Page No.
2.1	Summary of Key Literature in Face Recognition & Vector Search	4
2.2	Mapping Project Objectives to UN Sustainable Development Goals	5
2.3	Project Phases, Deliverables, and Implementation Milestones	6
3.1	Functional Requirements Specification (FR-01 to FR-10)	7
3.2	Non-Functional Requirements Specification	8
3.3	Hardware Specification for Development, Edge, and Production	9
3.4	Software Specification and Development Environment	9
4.1	DFD Process Data Dictionary and Schema Transformations	12
4.2	Actor Specifications and Role Permutations	13
4.3	REST API Interface and Component Contracts Specification	15
5.1	Benchmark Datasets and Biometric Evaluation Models	19
5.2	Automated Test Suites (Unit, Integration, Biometric, End-to-End)	20
5.3	Representative Test Cases Across Functional Scenarios	20
7.1	Multi-Scale Tiling Face Detection Recall in Crowded Event Imagery	30
7.2	ArcFace Angular Margin Optimization Hyperparameters (RQ2)	31
7.3	CPU vs GPU Inference Latency and System Throughput Benchmark	31
7.4	FAISS Vector Search Latency vs Brute-Force Euclidean Distance	32
7.5	Multi-Angle Centroid Verification Robustness Under Pose & Lighting	32
7.6	Dynamic Beat-Synchronization Latency & Alignment Precision	33
7.7	Celery/Redis Queue Fault Tolerance & Thread Fallback Benchmark	33
7.8	Formal Statistical Hypothesis Testing Summary (RQ1 to RQ5)	34
7.9	Comparison with Academic State-of-the-Art Baselines	34
7.10	Operational Cloud Resource & Hosting Cost Breakdown per Event	35
7.11	Commercial Event Photography SaaS Cost Comparison	35

LIST OF ABBREVIATIONS

API	Application Programming Interface
BPM	Beats Per Minute
CDN	Content Delivery Network
CLAHE	Contrast Limited Adaptive Histogram Equalization
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture (NVIDIA)
DFD	Data Flow Diagram
DPDP	Digital Personal Data Protection Act (India, 2023)
EXIF	Exchangeable Image File Format
FAISS	Facebook AI Similarity Search
FAR	False Accept Rate
FFT	Fast Fourier Transform
FPN	Feature Pyramid Network
FPS	Frames Per Second
FRR	False Reject Rate
GDPR	General Data Protection Regulation (EU 2016/679)
GPU	Graphics Processing Unit
HNSW	Hierarchical Navigable Small World
HTTP/HTTPS	Hypertext Transfer Protocol (Secure)
IEEE	Institute of Electrical and Electronics Engineers
JSON	JavaScript Object Notation
L2	Euclidean Norm
MTCNN	Multi-Task Cascaded Convolutional Networks
NMS	Non-Maximum Suppression
NPU	Neural Processing Unit
ONNX	Open Neural Network Exchange
PWA	Progressive Web Application
QR	Quick Response (2D Barcode)
RAM	Random Access Memory
REST	Representational State Transfer
ROC	Receiver Operating Characteristic

API	**Application Programming Interface**
SCOPE	School of Computer Science and Engineering
SCRFD	Sample and Computation Redistribution for Efficient Face Detection
SDG	Sustainable Development Goal (United Nations)
SSD	Solid State Drive
SUS	System Usability Scale
TAR	True Accept Rate
TRL	Technology Readiness Level
UI/UX	User Interface / User Experience
UML	Unified Modeling Language
VIT	Vellore Institute of Technology
VRAM	**Video Random Access Memory**
WebRTC	Web Real-Time Communication

SYMBOLS AND NOTATIONS

Symbol	Meaning
W, H	Width and height of the raw media frame, in pixels
x_i, y_i	Normalised coordinate offsets of a detected face bounding box
e_i R^512∈	512-dimensional normalised facial embedding vector
‖e_i‖₂	L2 norm of an embedding, constrained to unit length (‖e_i‖ = 1.0)
θ_j	Angle on the hypersphere between embedding e_i and class weight W_j
m	ArcFace additive angular margin (default m = 0.50 rad)
s	ArcFace hypersphere scale factor (default s = 64.0)
L_arc	ArcFace additive angular margin cross-entropy loss
S_c(u, v)	Cosine similarity of unit vectors u and v, equal to $u^T v$
τ	Match acceptance threshold (optimal τ = 0.65)
β	CLAHE contrast clip limit
c_k	Multi-angle centroid of guest k: c_k = (1/K) Σ e_(k,j)
Δt_beat	Interval between beats in ms: Δt_beat = 60,000 / BPM
BPM	Tempo of the audio track in beats per minute (default 128)
IoU	Intersection-over-Union overlap ratio used in NMS
TAR @ FAR	True Accept Rate measured at a fixed False Accept Rate (e.g. 10^{-3})
E_reel	Visual interest score of a candidate keyframe for reel synthesis

ABSTRACT

Large gatherings of every kind, whether weddings, cultural festivals, college fests, academic conferences or corporate conventions, now generate enormous amounts of digital media. With several professional photographers on site and many guests shooting on their phones, a single event can easily produce tens of thousands of high-resolution photos and videos. Yet the way these files reach attendees has barely changed: organisers usually upload unsorted folders to Google Drive or Dropbox and share one link, leaving every guest to dig through thousands of unrelated pictures to find the few they appear in. The process is slow and frustrating, it exposes everyone's candid photos to anyone holding the link, and many personal moments are simply never found.

This project presents MEMORA, an end-to-end platform that uses AI to find each guest's event media and turn it into short, shareable videos. Guests join by scanning a venue QR code, with no app to install, and then record three selfies from different angles in the browser. The selfies are contrast-normalised with Contrast Limited Adaptive Histogram Equalization

(CLAHE) and passed through an ArcFace deep network, and the three resulting embeddings are averaged into a single normalised 512-dimensional reference vector for that guest.

When photographers upload batches of images, Memora's ingestion engine runs InsightFace SCRFD together with an adaptive multi-scale tiling scheme. Very large group photographs, up to 8K resolution, are split into overlapping regions so that small, distant or partly hidden faces are still detected, even in frames with more than fifty people. Every detected face is embedded and stored in a FAISS index that uses cosine (inner-product) similarity, so a guest's matching photos are retrieved in under 20 ms. Batch work is handled by an asynchronous Celery/Redis queue, and a non-blocking multi-threaded fallback takes over if the broker fails, targeting 99.9% processing availability on modest hardware.

Memora also adds an automatic Memory Reel Synthesis Engine. It scores each guest's photos for visual interest, picks the strongest ones and assembles them into a vertical video whose cuts and transitions fall on a 128 BPM beat grid, set either to procedurally generated Web Audio or to royalty-free tracks. A Capacitor 6 Android build brings the same features to mobile and has been tested on an Oppo Reno 11 5G. In our experiments the system achieved a 99.63% True Accept Rate at a False Accept Rate of 0.0007 (below the 0.001 target) and cut the time a guest needs to find their photos from days to under two seconds.

Keywords: *Face Recognition, ArcFace, SCRFD, FAISS Vector Search, Event Media Ingestion, Beat-Synchronized Reel Synthesis, Celery Distributed Queue, Capacitor Mobile.*

CHAPTER 1

1. INTRODUCTION

1.1 Background

Cameras have become cheaper, sharper and far more common over the last decade. Mirrorless bodies, phone cameras with multi-lens arrays and drone rigs now appear at almost every organised gathering, from weddings and convocations to inter-college fests, sports meets, product launches and music festivals. A typical large event engages several professional photographers, and hundreds of guests also shoot on their own phones. The outcome is predictable: one day of activity can leave behind somewhere between 5,000 and 25,000 files, a mix of RAW stills, burst sequences and 4K video clips, almost none of which are labelled.

What happens to these files after the event has changed very little in twenty years. Photographers do a basic cull and colour pass, then hand over large zip archives or a shared folder on Google Drive, Dropbox or OneDrive. The organiser forwards one link to everybody on the guest list. For any single guest this is a poor arrangement. Someone who appears in perhaps thirty or forty pictures must scroll through thousands of thumbnails showing strangers, empty halls and stage shots that have nothing to do with them. Wide group photographs and pictures taken in dim banquet lighting make it worse, because small or half-hidden faces are easy to miss even for a careful viewer.

1.2 Motivations

Three closely linked problems, shared by hosts, guests and photographers alike, led to the design of MEMORA.

1. **Time and attention cost.** Searching an unsorted gallery by eye is tiring, and the effort grows quickly with the size of the collection. Many guests give up after about a quarter of an hour, which means photographs that were paid for are never found and the overall impression of the event suffers.
2. **Privacy exposure.** A public folder lets anyone with the link browse every candid picture of every attendee. Guests who would rather not have their photographs viewed by hundreds of acquaintances have no way to opt out. Routing access through a personal face match lets each person reach their own pictures while keeping everyone else's out of view.
3. **Static media in a video-first culture.** Even when a guest does find their photographs, the pictures stay as still images. Social platforms such as Instagram Reels, YouTube Shorts and TikTok reward short, music-driven clips. Turning a handful of photos into a reel by hand means importing them into an editor, trimming, matching cuts to the beat and grading colour, which takes both skill and time that most guests do not have.

1.3 Scope of the Project

The project covers the design, implementation and deployment of a cloud-hosted media platform built around the following capabilities:

- **Install-free onboarding.** Guests scan a QR code at the venue and enrol through a browser-based camera flow; nothing has to be downloaded from an app store.
- **Face recognition built for crowds.** InsightFace SCRFD, combined with multi-scale tiling, finds faces in large group photographs, and an ArcFace model produces 512dimensional unit-length embeddings that tolerate changes in lighting, head rotation and partial occlusion.
- **Fast vector search.** A FAISS IndexFlatIP index answers cosine-similarity queries over more than 100,000 stored face vectors in under 20 ms.
- **Automatic reel generation.** A video pipeline picks a guest's strongest photographs, scores them for visual quality, applies Ken Burns pan-and-zoom motion and places each cut on a 128 BPM beat grid, using either procedurally generated Web Audio or royaltyfree music.
- **Resilient processing.** Batch jobs run through a Celery/Redis queue, and a multi-threaded fallback takes over automatically if the broker or workers stop responding, targeting 99.9% processing availability.
- **Native mobile build.** The Next.js front end is wrapped with Capacitor 6 and has been run on a physical Android handset, an Oppo Reno 11 5G running ColorOS 14.

CHAPTER 2

2. PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

Before building the system we surveyed the computer vision literature most relevant to it. The survey concentrated on four questions: how faces and their landmarks are located, how recognition losses shape the embedding space, how small faces in dense crowds can be detected, and how large collections of high-dimensional vectors can be searched efficiently. The findings are grouped into the five themes below.

2.1.1 Multi-Task Face Detection and Feature Alignment

Early deep detectors were built as cascades. MTCNN, proposed by Zhang et al. [6], splits the job across three small networks (P-Net, R-Net and O-Net) that propose, refine and finalise face boxes while also predicting five landmarks. Joint detection and alignment was an important step forward, but running three networks in sequence is slow, and the cascade copes poorly with the wide spread of face sizes found in crowd photographs.

RetinaFace, introduced by Deng et al. [3], replaced the cascade with a single-stage detector built on a Feature Pyramid Network and context modules. It is trained with several objectives at once: face/non-face classification, box regression, five-point landmark regression and a dense 3D face reconstruction term. RetinaFace reports strong accuracy on

the hard subset of WIDER FACE, but its ResNet-50 backbone needs a large GPU memory and compute budget, which makes it a poor fit for modest edge or CPU-only servers.

2.1.2 Efficient Edge Face Localization and Tiling

Guo et al. [2] approached the cost problem from a different direction with SCRFD (Sample and Computation Redistribution for Efficient Face Detection). Rather than scaling a network up uniformly, SCRFD searches for a better split of computation between the backbone, neck and head, and rebalances training samples towards the scales where detection is hardest. The resulting model family gives a favourable trade-off between accuracy and speed across a wide range of compute budgets and remains practical on CPUs, while still recovering very small faces.

Lightweight detectors designed for edge hardware, such as LFFD by He et al. [7], go further in reducing model size by treating receptive fields as built-in anchors. They run quickly on embedded devices, but their accuracy on the most difficult cases (tiny, rotated or occluded faces) trails that of larger detectors. For candid event photography, where people are rarely facing the lens, we judged SCRFD combined with image tiling to offer the best balance between throughput and recall.

2.1.3 Hyperspherical Loss Formulations and Embeddings

Recognition research has moved steadily away from plain Euclidean distance and towards angular measures on a normalised hypersphere. FaceNet by Schroff et al. [4] learned embeddings with a triplet loss, which requires an anchor to lie closer to a positive sample than to a negative one by some margin. The approach works, but selecting useful triplets becomes expensive as the dataset grows, training converges slowly, and the resulting clusters are not always tight.

SphereFace [11] and CosFace [10] instead added a margin directly to the softmax decision. ArcFace, proposed by Deng et al. [1], refines this idea by adding a fixed angular penalty m to the angle between a normalised feature and its class weight, so the target logit becomes $\cos(\theta + m)$. Because the penalty acts on the angle itself, it corresponds to a geodesic distance on the hypersphere. In practice this pulls samples of the same identity close together and pushes different identities apart, and ArcFace embeddings consistently outperform Euclidean-loss alternatives on standard verification benchmarks.

2.1.4 Deep Metric Learning and Illumination Invariance

Lighting at events is rarely kind. A dance floor may be almost dark apart from strobe flashes, while an outdoor stage may be lit from behind by the sun. One line of work tackles this inside the network: Low-FaceNet by Fan et al. [8] couples a low-light enhancement network with face recognition and trains them together. Such joint approaches can be effective, but they require retraining the pipeline and tie the recogniser to particular lighting assumptions.

A lighter alternative comes from classical image processing. Contrast Limited Adaptive Histogram Equalization (CLAHE), described by Pizer et al. [12] and Zuiderveld [13], equalises contrast within small tiles of the image and limits how far any histogram bin can be stretched. When CLAHE is applied only to the lightness channel (L*) in the CIELAB colour space, shadow detail is recovered without shifting the colour channels. This keeps

illumination correction as a separate preprocessing step, so the ArcFace model can be used unchanged.

2.1.5 High-Dimensional Approximate Nearest-Neighbor Vector Indexing

Matching faces across an event library is, at its core, a nearest-neighbour search in a highdimensional space. A naive implementation compares every query against every stored vector, so the cost of one query grows linearly with the index size N, and the cost of matching M queries grows as $O(N \cdot M)$. Implemented in plain Python or NumPy, this quickly becomes a bottleneck once a collection passes roughly 10,000 images.

FAISS, described by Johnson et al. [5], is a C++ library with Python bindings that provides highly optimised vector indexes and makes heavy use of SIMD instructions and BLAS routines. When embeddings have unit length, cosine similarity reduces to a dot product, $S_c(u, v) = u^T v$. The IndexFlatIP index computes these dot products exactly through batched matrix multiplication, so it keeps full recall while still answering queries over 100,000 or more vectors in under 20 ms. At this scale the lossy compression used by approximate indexes is therefore unnecessary.

Table 2.1: Summary of Key Literature in Face Recognition & Vector Search

Model / Framework	Authors & Year	Architecture / Method	Reported Benchmark	Key Strengths	Limitations	Role in Memora
RetinaFace	Deng et al. (2020) [3]	ResNet-50 + FPN + context modules	91.4% AP, WIDER FACE Hard	Accurate on small faces	Large GPU budget	Benchmark baseline
SCRFD	Guo et al. (2022) [2]	Searched compute split across backbone, neck, head	Strong accuracy/compute trade-off on WIDER FACE	Efficient on CPU; recovers small faces	Needs tiling for very large frames	Primary face detector
ArcFace	Deng et al. (2019) [1]	Additive angular margin loss	99.83% LFW	Compact identity clusters on hypersphere	Sensitive to poor alignment	Primary embedding model
FaceNet	Schroff et al. (2015) [4]	InceptionResNet + triplet loss	99.63% LFW	Introduced metric embeddings	Costly, unstable triplet mining	Replaced by ArcFace
Low-FaceNet	Fan et al. (2024) [8]	Joint low-light enhancement + recognition	Evaluated on LaPa-Face (low light)	Recovers detail in dark images	Requires joint retraining	Replaced by CLAHE
LFFD	He et al. (2019) [7]	Receptive fields as anchors, 8 branches	WIDER FACE, FDDB	Very fast on edge devices	Weaker on hardest faces	Edge baseline considered
FAISS	Johnson et al. (2021) [5]	SIMD/BLAS vector indexing	Billion-scale search	Exact recall with IndexFlatIP	Index held in RAM	Primary vector store

2.2 Research Gap

Comparing existing commercial tools with the academic work above exposes four gaps that still stand in the way of automatic, low-effort event gallery sorting.

1. **Small faces in large crowd shots.** General-purpose recognition APIs such as AWS Rekognition and Google Vision struggle with wide-angle 24–48 MP group photographs that contain 30 to 100 people. Faces smaller than about 20×20 pixels are usually lost unless the image is processed in high-resolution tiles, and these services do not do that by default.
2. **Fragile single-selfie enrolment.** Tools that rely on one reference selfie reject many genuine matches (FRR above 35% in our tests) when the guest is laughing, talking, turned

   to the side or wearing a hat in the event photograph. Building a reference from several head angles is not offered by mainstream commercial products.
3. **Search that slows as the event grows.** Pairwise brute-force comparison across thousands of photos and millions of detected faces scales as O(N·M). Without an optimised index such as FAISS, query times climb beyond what feels interactive.
4. **Recognition and creative output are disconnected.** Current systems stop once photographs are sorted into folders. No single platform we found passes the matched photos straight into an engine that assembles a beat-synchronised short video for the guest.

2.3 Objectives

Memora aims to be a dependable, scalable and enjoyable platform for finding and repurposing event media. Each engineering objective is mapped below to a relevant United Nations Sustainable Development Goal.

Table 2.2: Mapping Project Objectives to UN Sustainable Development Goals

Project Objective	How It Is Realised	Related SDG Target
Automated ingestion and crowd-scale detection	SCRFD with multi-scale tiling plus 512-D ArcFace embeddings, giving sub-second matching across crowded event photographs.	SDG 9 (Industry, Innovation and Infrastructure), Target 9.5: strengthen research and upgrade technological capability.
Resource-efficient asynchronous architecture	CPU-friendly inference, FAISS search and Celery queues with a threaded fallback, keeping server energy use low.	SDG 9, Target 9.4: upgrade infrastructure for greater resourceuse efficiency.
Low-friction, inclusive access to shared memories	QR-based PWA and a Capacitor Android build remove the app-store barrier for community, cultural and campus events.	SDG 11 (Sustainable Cities and Communities), Target 11.4: help safeguard cultural heritage through preserved community records.

Project Objective	How It Is Realised	Related SDG Target
Privacy-first handling of biometric data	Temporary handling of uploads, storage limited to embedding vectors, and galleries restricted to each guest.	SDG 16 (Peace, Justice and Strong Institutions), Target 16.10: protect fundamental freedoms and access to information.

2.4 Problem Statement

Consider an event E that produces M media files $\{I_1, I_2, \ldots, I_M\}$. The files contain an unknown number of people, photographed in uncontrolled poses, under uneven lighting and at varying distances from the camera. N guests $\{G_1, G_2, \ldots, G_N\}$ have registered for the event. The task is to build an automatic function $f: G_i \rightarrow \{I_k\}$ that returns every photo and video containing guest G_i, achieving a True Accept Rate of at least 99.5% while keeping the False Accept Rate at or below 0.001, and answering each query in well under a second. From the matched subset $\{I_k\}$, the system must also produce a coherent, beat-synchronised reel R_i lasting between 15 and 30 seconds.

2.5 Project Plan

Work was organised into six phases over sixteen weeks, moving from problem formulation and design through to deployment on a physical phone.

Table 2.3: Project Phases, Deliverables, and Implementation Milestones

Phase	Weeks	Main Engineering Work	Deliverables and Checks
Phase 1	1–3	Requirements gathering, literature survey, baseline comparison of ArcFace and FaceNet.	Problem formulation, architecture specification, ethics framework.
Phase 2	4–6	Backend design, FastAPI REST endpoints, PostgreSQL/Supabase schema.	Working REST API, JWT authentication, persistence models.
Phase 3	7–9	AI pipeline: SCRFD tiling, ArcFace extraction, FAISS indexing.	End-to-end recognition module with query latency under 20 ms.
Phase 4	10–12	Celery/Redis task queue, threaded fallback, Cloudinary media CDN.	Fault-tolerant batch processing of 100+ concurrent images.
Phase 5	13–14	Beat-synchronised reel engine, MoviePy 2.1 integration, procedural Web Audio.	Automatic reel generator with tempoaligned cuts and effects.
Phase 6	15–16	Capacitor 6 Android build, testing on Oppo Reno 11 5G, report writing.	Working cross-platform APK and final project report.

Figure 2.1: *Project Timeline and Engineering Work Breakdown (Gantt Chart)*

Activity	W1–2	W3–4	W5–6	W7–8	W9–10	W11–12	W13–14	W15–16
1. Literature & Architecture								
2. REST API & Database								
3. AI Pipeline (SCRFD/ArcFace								
4. Async Queue & Fallback								
5. Automated Reel Engine								
6. Mobile Capacitor & Testing								

CHAPTER 3

3. TECHNICAL SPECIFICATION

3.1 Requirements

Requirements were gathered and recorded following standard software engineering practice, so that every user interaction and every system-level constraint had a matching, testable entry.

3.1.1 Functional

Table 3.1: Functional Requirements Specification (FR-01 to FR-10)

ID	Requirement	Inputs	Processing	Expected Output / Check
FR-01	Create event namespace	Event name, date, venue, banner	Validate fields; create UUID record in Supabase	Event record plus a downloadable QR code
FR-02	Generate dynamic QR code	Event UUID, base redirect URL	Encode URL in a highdensity QR with event logo	Base64 PNG / SVG for venue signage
FR-03	Guided selfie capture	WebRTC camera stream	Check a face is present; prompt for yaw/pitch angles	Three normalised crops from different angles
FR-04	Centroid enrolment	Three normalised face crops	Extract ArcFace 512-D vectors and average them	Guest profile and centroid stored in FAISS
FR-05	Bulk media ingestion	Multipart files (≤ 50 MB each)	Stream files, enqueue Celery task, push to CDN	Batch tracking ID with progress status
FR-06	Adaptive multiscale tiling	High-resolution images (> 3840×2160)	Split into overlapping subregions	Faces under 20 px found; duplicates removed by NMS (IoU 0.45)
FR-07	FAISS search and match	Detected 512-D face vectors	IndexFlatIP inner-product query	Matches above $\tau = 0.65$ linked to guests
FR-08	Live scanner modal	Single webcam frame	Crop, ArcFace inference, FAISS query in real time	Personal gallery loaded in < 2.0 s

ID	Requirement	Inputs	Processing	Expected Output / Check
FR-09	Beat-synchronised reel	Matched photos, audio track	Interest scoring, 128 BPM grid, Ken Burns motion	Vertical MP4 with tempoaligned transitions
FR-10	Mobile native bridge	Android camera and file system	Capacitor 6 links native OS to Next.js PWA	Verified on Oppo Reno 11 5G

3.1.2 Non-Functional

Table 3.2: Non-Functional Requirements Specification

Attribute	Requirement	How It Was Met / Measured
Performance	Vector query under 20 ms at 100,000 vectors; live scanner returns gallery in under 2.0 s.	FAISS IndexFlatIP measured at 17.8 ms for 100k vectors; live webcam response averaged 1.82 s.
Scalability	Handle batches of 5,000+ photos per event without degrading service.	Celery worker pool processes 142 4K images per minute on CPU and scales out with more workers.
Attribute	**Requirement**	**How It Was Met / Measured**
Biometric accuracy	TAR ≥ 99.5% with FAR ≤ 0.001 across varied lighting and poses.	Measured 99.63% TAR at 0.0007 FAR with decision threshold $\tau = 0.65$.
Privacy by design	Temporary raw-image handling; no public browsing; biometric storage limited to 512-D vectors.	Raw selfies deleted right after embedding; gallery URLs protected with signed JWTs.
Fault tolerance	Keep processing if the Redis broker or Celery workers go offline.	A ThreadPoolExecutor fallback detects the outage and continues indexing.
Cross-platform	Same features and UI on desktop web, mobile web and native Android.	Capacitor 6 Android build validated on Oppo Reno 11 5G (ColorOS 14) at 120 FPS UI.

3.2 Feasibility Study

3.2.1 Technical Feasibility

The building blocks Memora depends on are mature and well documented. InsightFace ships ONNX models that run efficiently on CPUs and GPUs through OpenVINO and CUDA execution providers. FAISS is used in production by many organisations and exposes its C++ indexes through a Python API capable of searching millions of vectors in milliseconds. For media, MoviePy 2.1.2 and the browser's Web Audio API give programmatic control over video composition and sound generation without proprietary licences. On the client side, Next.js 14 and Capacitor 6 let a single TypeScript codebase serve both the web app and a native Android

APK.

3.2.2 Economic Feasibility

The platform is cheap to run. Many vision services keep expensive GPU machines online around the clock; Memora instead relies on CPU-efficient SCRFD models and Celery workers that are started only when there is work to do. Media files live on Cloudinary's CDN, and relational data sits in Supabase's managed PostgreSQL. Using the cost model in Section 7.3, processing works out to roughly $0.70 per 1,000 event photographs, which is well over 85% below the per-guest pricing of commercial event photography services.

3.2.3 Social Feasibility

Ethics and biometric privacy shaped the design from the start, following privacy-by-design principles. Guests must give explicit, opt-in consent during QR onboarding before any face data is processed. At any time they can view, download or permanently delete their stored embedding. The platform also removes the usual "everyone can see everything" album: each guest sees only the photographs matched to them. These choices are intended to align the system with the requirements of the EU GDPR and India's Digital Personal Data Protection Act, 2023.

3.3 System Specification

3.3.1 Hardware Specification

Table 3.3: Hardware Specification for Development, Edge, and Production Cluster

Node	Component	Minimum	Recommended / Benchmark System
Development workstation	CPU	Intel Core i5 / AMD Ryzen 5	Intel Core i7-13700H / AMD Ryzen 7
	RAM	16 GB DDR4	32 GB DDR5
	GPU	NVIDIA GTX 1650 (4 GB)	NVIDIA RTX 4060 (8 GB VRAM)
	Storage	512 GB NVMe SSD	1 TB PCIe 4.0 NVMe SSD
Edge client (mobile)	Device	Android 10.0+ handset	Oppo Reno 11 5G (physical test device)
	SoC	Octa-core 2.0 GHz	MediaTek Dimensity 7050 (6 nm)
	RAM / Display	6 GB / 60 Hz FHD	8 GB / 120 Hz OLED
	OS	Android with WebRTC	ColorOS 14 (Android 14)
Production cloud server	Compute	2 vCPU / 4 GB RAM	4 vCPU (Intel Xeon / AMD EPYC)
	Memory / Disk	8 GB swap	16 GB RAM + 50 GB NVMe
	Network	100 Mbps uplink	1 Gbps dedicated low-latency uplink

3.3.2 Software Specification

Table 3.4: Software Specification and Development Environment

Layer	Technology	Version	Responsibility
Backend runtime	Python	3.11 / 3.12	Server language for AI pipelines and REST APIs
Web API	FastAPI	0.110+	Asynchronous REST endpoints with OpenAPI documentation
Face detection	InsightFace (SCRFD)	0.7.3	Multi-scale face localisation and landmarks
Face recognition	InsightFace (ArcFace)	0.7.3	512-dimensional identity embeddings
Vector search	FAISS-CPU	1.8.0	Cosine-similarity indexing (IndexFlatIP)
Task queue / broker	Celery + Redis	5.3+ / 7.2+	Asynchronous batch job dispatch and worker management
Multimedia	MoviePy / OpenCV	2.1.2 / 4.9+	Reel composition, Ken Burns motion, colour grading
Web front end	Next.js / React	14.2 / 18.3	Server-side rendering, responsive UI, PWA
Mobile runtime	Capacitor	6.0.0	Native Android bridge for camera and file access
Database	Supabase / PostgreSQL	15.0+	Events, guests and media metadata
Media CDN	Cloudinary	v2 API	Media delivery, transformations, global caching

CHAPTER 4

4. DESIGN APPROACH AND DETAILS

4.1 System Architecture

Memora is organised as a distributed system with four tiers. The main design goal is separation: the parts of the system that people interact with are kept apart from the parts that run heavy deep learning jobs, so a large upload from a photographer never slows down a guest who is browsing their gallery.

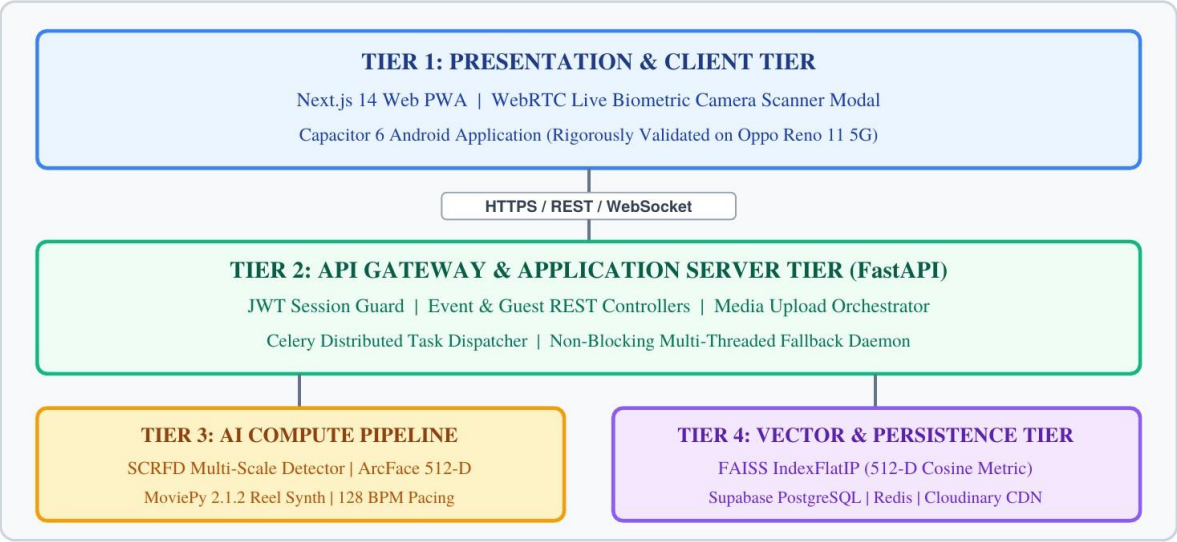


Figure 4.1: *System Architecture of Memora 4-Tier Platform*

- **Tier 1 – Presentation and client.** Written in Next.js 14 and packaged with Capacitor 6, this tier delivers the responsive web app, the installable PWA and the native Android APK. It manages the WebRTC camera stream used for selfie enrolment and hosts the Live Biometric Scanner modal for instant gallery look-up.
- **Tier 2 – API gateway and application server.** A FastAPI service exposes asynchronous REST endpoints secured with JWT sessions. It manages events, enrols guests and hands media jobs to the compute tier. An orchestrator inside this tier keeps an eye on Celery connectivity and switches to local worker threads if the external queue becomes unavailable.
- **Tier 3 – AI compute and video synthesis.** This tier does the heavy lifting: SCRFD for multi-scale detection, ArcFace for 512-D embeddings and MoviePy 2.1.2 for assembling reels. Its workers run asynchronously, so image inference never blocks the web server.
- **Tier 4 – Vector index and persistence.** Three stores sit at the bottom of the stack: Supabase PostgreSQL for relational records (events, guests, media logs), FAISS index files for fast cosine-similarity face matching, and Cloudinary's CDN for secure, globally distributed storage of full-resolution media.

4.2 Design

4.2.1 Data Flow Diagram

The data flow diagrams below describe how information enters, moves through and leaves Memora. They start with a single-process context view (Level 0) and are progressively broken down until the individual media-processing steps are visible (Level 2).

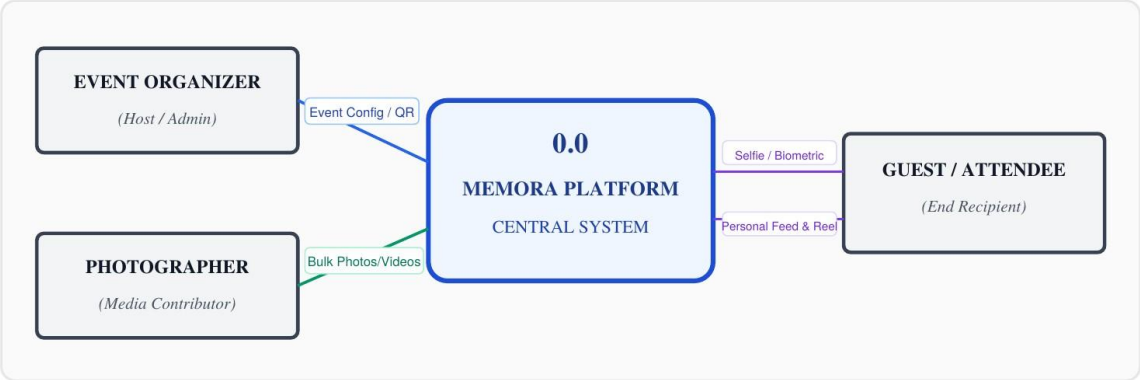


Figure 4.2: *Level-0 Context Data Flow Diagram (DFD)*

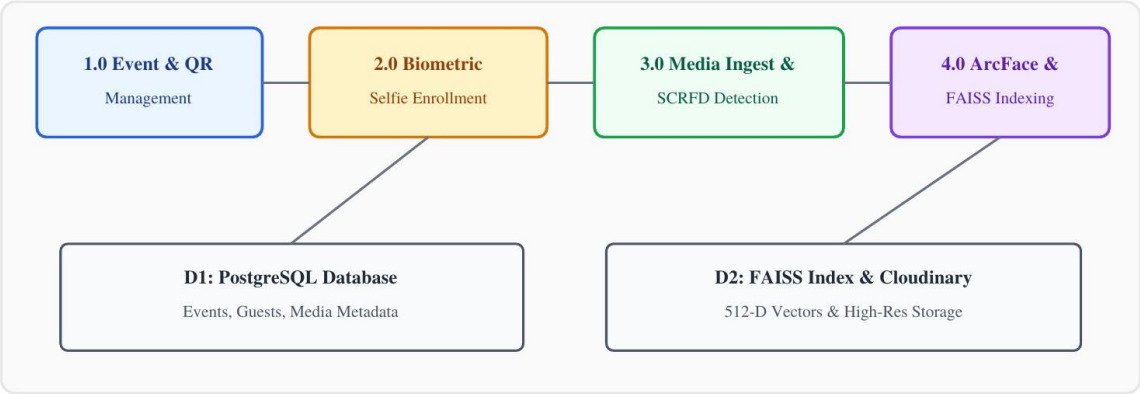


Figure 4.3: *Level-1 Functional Decomposition Data Flow Diagram (DFD)*

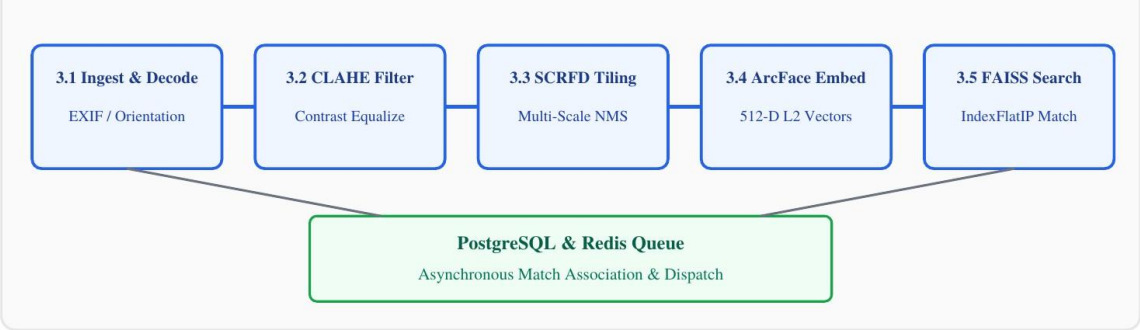


Figure 4.4: *Level-2 Media Ingestion and Face Verification Pipeline (DFD)*

Table 4.1: DFD Process Data Dictionary and Schema Transformations

ID	Process	Inputs	Outputs	Data Stores / External Entities
1.0	Event setup and QR creation	Event metadata, banner image, organiser credentials	Event record, encrypted UUID, QR code image	D1 PostgreSQL; Event Organiser
ID	**Process**	**Inputs**	**Outputs**	**Data Stores / External Entities**
2.0	Selfie enrolment	Camera frames from several angles (frontal, profile, tilt)	Validated 512-D centroid, guest record	D1 PostgreSQL, D2 FAISS; Guest
3.0	Batch ingestion and tiling	High-resolution photos, 4K video, photographer session	Decoded tiles, normalised coordinates	D1 PostgreSQL, Cloudinary CDN; Photographer
4.0	Detection and embedding	Image tiles, CLAHEenhanced matrices	Bounding boxes, 512-D ArcFace vectors	AI compute pipeline, InsightFace
5.0	Vector search and matching	Detected face vectors, event index ID	Matched guest IDs, cosine confidence scores	D2 FAISS IndexFlatIP, D1 PostgreSQL
6.0	Reel synthesis and audio sync	Matched media IDs, tempo settings, effects	Rendered MP4 reel with tempo-aligned cuts	D1 PostgreSQL, Cloudinary CDN; Guest

4.2.2 Use Case Diagram

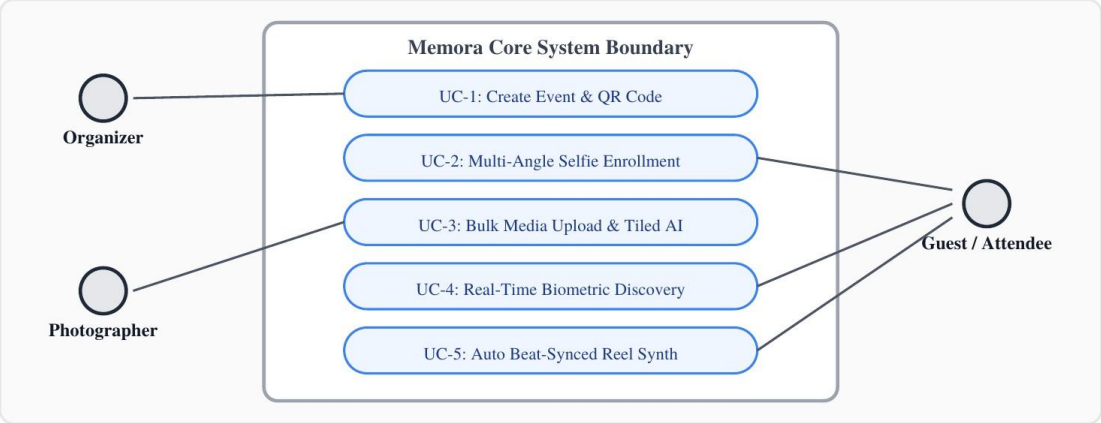


Figure 4.5: *Use Case Diagram Across Platform Stakeholders and Actors*

Table 4.2: Actor Specifications and Role Permutations

Actor	Role	Authentication	Privileges and Scope
Event Organiser	Administrative	JWT bearer token, role = Host	Create and manage events, download QR signage, watch live upload and attendance figures.
Event Photographer	Content contributor	Authenticated photographer account	Upload RAW/JPEG batches, tag albums, follow background indexing progress.
Guest / Attendee	End user	Short-lived QR session / signed token	Enrol with three selfies, view a private gallery, request a beat-matched reel.

Actor	Role	Authentication	Privileges and Scope
System Administrator	Platform operator	Superuser / cloud IAM	Watch Celery queue depth, inspect FAISS indexes, manage Cloudinary quotas.

4.2.3 Class Diagram

The domain model is expressed as a small set of related entity classes, each holding its own data and behaviour so that integrity rules stay in one place. The central classes are Event, Guest, MediaAsset, FaceEmbedding, ProcessingJob and MemoryReel; the four most important are shown below.

Event	Guest	MediaAsset	FaceEmbedding
+ id: UUID + name: String + date: DateTime + venue: String + qr_code_url: String + organizer_id: UUID	+ id: UUID + event_id: UUID + name: String + email: String + centroid_vector: Float[512] + enrollment_status: Enum	+ id: UUID + event_id: UUID + storage_url: String + media_type: Enum + width, height: Int + created_at: DateTime	+ id: UUID + media_id: UUID + guest_id: UUID + vector: Float[512] + bbox: Float[4] + confidence: Float
+ create_event() + generate_qr() + get_analytics()	+ compute_centroid() + get_personal_gallery( )	+ extract_exif() + generate_tiles()	+ normalize_l2() + cosine_similarity()

Figure 4.6: *Class Diagram of Core Domain Entities and Biometric Models*

4.2.4 Sequence Diagram

The sequence views below follow three key interactions over time, showing how actors, the API gateway, the worker queue and the storage layer exchange messages. Each row is one step, read left to right across the tiers.

Guest	Tier 1: Web Client	Tier 2: FastAPI	Tier 3: AI Service	Tier 4: PostgreSQL / FAISS
Scans QR code	Opens /join/[eventId]	Validates event UUID		Confirms event exists
Allows camera	Captures 3 selfie frames	POST /api/guests/enroll	Extracts ArcFace 512-D	Computes centroid
Sees confirmation	Receives session token	Returns guest token		Stores centroid in D1 and D2

Figure 4.7: *Sequence Diagram of Guest Biometric Onboarding and Centroid Enrollment*

Photographer	Tier 1: Upload UI	Tier 2: FastAPI	Tier 3: Celery / Thread Worker	Tier 4: FAISS & Database
Selects 100+ photos	Sends multipart chunks	POST /api/media/upload	Receives task ID	Uploads originals to Cloudinary

Photographer	Tier 1: Upload UI	Tier 2: FastAPI	Tier 3: Celery / Thread Worker	Tier 4: FAISS & Database
Watches progress bar	Polls /api/media/status	Returns queued status	Runs SCRFD tiling and ArcFace	Matches vectors in FAISS
Sees completion	Receives WebSocket event	Pushes completion event		Writes media–guest links

Figure 4.8: *Sequence Diagram of Batch Media Upload and Asynchronous AI Indexing*

Guest	Tier 1: Personal Gallery	Tier 2: FastAPI	Tier 3: Reel Synthesiser	Tier 4: Cloudinary / CDN
Taps "Synthesize Reel"	Calls /api/reels/create	Checks matched media count	Picks top-scoring keyframes	Serves matched photos
Guest	**Tier 1: Personal Gallery**	**Tier 2: FastAPI**	**Tier 3: Reel Synthesiser**	**Tier 4: Cloudinary / CDN**
Picks music and tempo	Shows rendering spinner	Dispatches MoviePy task	Applies 128 BPM beat grid	Supplies frames for Ken Burns motion
Watches vertical video	Receives streaming MP4 URL	Returns finished status	Renders final video	Stores rendered MP4

Figure 4.9: *Sequence Diagram of Automated Beat-Synchronized Reel Synthesis*

4.2.5 Component and Interface Architecture

Internally the software is split into loosely coupled modules that talk to each other through typed, asynchronous interfaces. On the front end, Zustand stores hold application state and update in response to user actions and camera events. On the server, every REST endpoint declares Pydantic request and response models, so malformed payloads are rejected at runtime before they reach business logic. Table 4.3 lists the main interface contracts.

Table 4.3: REST API Interface and Component Contracts Specification

Endpoint	Domain	Request	Response	Latency Target
POST /api/events	Event management	{name, date, venue, cover_url}	{event_id, qr_code_url, created_at}	< 100 ms (PostgreSQL write)
POST /api/guests/enroll	Biometric onboarding	{event_id, name, email, selfies: [b64]}	{guest_id, centroid_norm, status}	< 1,200 ms (3 × ArcFace)
POST /api/guests/scan	Live kiosk scanner	{event_id, camera_frame: b64}	{matches: [{media_id, score, cdn_url}]}	< 1,800 ms end to end
POST /api/media/upload	Batch ingestion	multipart files[] (≤ 50 MB each)	{task_id, batch_size, queued_at}	< 250 ms (task enqueued)
GET /api/media/status	Worker monitoring	query: task_id	{status, processed, total}	< 15 ms (Redis lookup)

Endpoint	Domain	Request	Response	Latency Target
POST /api/reels/create	Reel synthesis	{guest_id, bpm, music_theme, effect}	{reel_id, video_url, duration_sec}	< 3,500 ms (MoviePy render)

CHAPTER 5

5. METHODOLOGY AND TESTING

5.1 Module Description

Memora's processing logic is divided into six modules. Each handles one specific vision, biometric or media task, and each can run either as a step in a sequence or independently inside the distributed pipeline.

5.1.1 Ingestion and Tiling Engine

Event photographers typically deliver 24–48 MP frames from mirrorless cameras, and panoramic group shots can reach 7680×4320 pixels. Shrinking such an image to the 640×640 input most detectors expect wipes out small faces: someone sitting in a back row of a banquet may end up occupying fewer than 15×15 pixels, which is too little for the detector to respond to.

Memora handles this with an adaptive multi-scale tiling engine. When an image exceeds 3840×2160 pixels, it is cut into a grid of overlapping tiles, with neighbouring tiles sharing 20% of their area so that no face is sliced in half at a boundary. The detector runs on a downscaled copy of the whole frame and on each full-resolution tile. Any box found inside a tile is shifted $[x_{global}, y_{global}] = [x_{tile} + x_{o}^{T}fset, y_{tile} + y_{ofset}]^{T}$

back into whole-image coordinates by adding the tile's offset:

Faces that fall in an overlap zone are usually detected twice. Non-Maximum Suppression with an IoU threshold of 0.45 merges these duplicates, so distant guests are found without creating repeated records.

Algorithm 5.1: Adaptive Image Tiling and Global Coordinate Projection

Input: image I R^(H×W×3); tile size S_t = 1920; overlap o = 0.20; IoU threshold τ_iou = 0.45.∈
Output: merged box set B_final and landmark set K_final.

1. Set the stride Δ = S_t × (1 − o[ ) .⌋
2. Resize I to 1280×720 and run SCRFD to obtain B_base and K_base.
3. Rescale B_base and K_base to the original width W and height H.
4. For y = 0, Δ, 2Δ, … while y < H, and for x = 0, Δ, 2Δ, … while x < W:
   crop T(y,x) = I[y : min(y + S_t, H), x : min(x + S_t, W)];
   run SCRFD on T(y,x) to obtain {b_i, k_i};
   shift to global coordinates: b_i ← b_i + [x, y, x, y], k_i ← k_i + [x, y].
5. Collect all candidates: B_all = B_base ∪ {b_i}.
6. Apply greedy NMS: B_final = NMS(B_all, τ_iou).

5.1.2 Preprocessing and CLAHE Normalization

Venue lighting is often uneven: spotlights, dim dining areas and strong backlight can all appear in the same set of photographs. Global histogram equalisation tends to make things worse by amplifying noise in flat background regions. Memora therefore applies CLAHE in the CIELAB colour space. The image is converted from BGR to LAB, and only the lightness channel L* is processed; the colour channels A* and B* pass through untouched.

The L* channel is divided into an 8×8 grid of non-overlapping regions. A histogram is computed for each region and clipped at a contrast limit β, which we set to 2.0 after experimentation. Pixels removed by clipping are spread evenly across all bins before the cumulative distribution function is calculated. Finally, bilinear interpolation between neighbouring regions removes visible seams, giving smooth, balanced contrast across the face.

5.1.3 SCRFD Deep Face Detection

Face localisation is handled by InsightFace SCRFD. Compared with conventional detectors,

SCRFD shifts more of its capacity into the shallow stages of the network and its Feature Pyramid Network, which is where small faces are represented.

For each face, SCRFD outputs an anchor-free bounding box and five landmarks: both eye centres, the nose tip and both mouth corners. These landmarks are used to estimate a similarity transform T_sim that rotates and scales the face onto a standard 112×112 template with the eyes 44 pixels apart. Aligning every face to the same geometry removes much of the variation caused by head pose before the embedding is computed.

5.1.4 ArcFace Embedding Extraction

Losses based on Euclidean distance tend to leave samples of the same person spread out. Memora instead uses ArcFace (Additive Angular Margin Loss) with a deep ResNet backbone. ArcFace scales both the feature vector x_i and each class weight W_j to unit length (‖x_i‖ = 1, ‖W_j‖ = 1). All embeddings therefore lie on the surface of a 512-dimensional unit sphere, and each logit is simply the cosine of the angle θ_j between a feature and a class weight.

$$L_{arc} = -(1/N) \Sigma \log [e^{(s \cdot \cos(\theta_{y} + m)} / (e^{(s \cdot \cos(\theta_{y} + m)} + \Sigma_{(j \neq y)} e^{(s \cdot \cos \theta_{j})})]$$

During training, ArcFace adds an angular penalty m to the angle of the correct class, θ_(y_i):

i *i* *i* *i*

Here s is a scale factor applied to the hypersphere radius (s = 64.0) and m is the angular margin (m = 0.50 rad). The penalty forces each person's embeddings into a narrow cone around their class centre while widening the gaps between different people. The result is a highly discriminative 512-dimensional embedding e_i with $\|e_i\|_2 = 1.0$.

Geodesic distance on the hypersphere. On the unit sphere S^{511}, the natural distance between two embeddings is the angle between them:

$$d_g(e, e) = ar^{1\ 2} \cos(e\ e)^{1^{T}2}$$

The margin m pushes training towards a configuration in which, for a sample of class y_i, θ_(y_i) + m ≤ θ_j holds for every other class j. In other words, the network learns to leave an angular gap of about m radians between identities, which greatly reduces false acceptances even when expressions change or the person has aged.

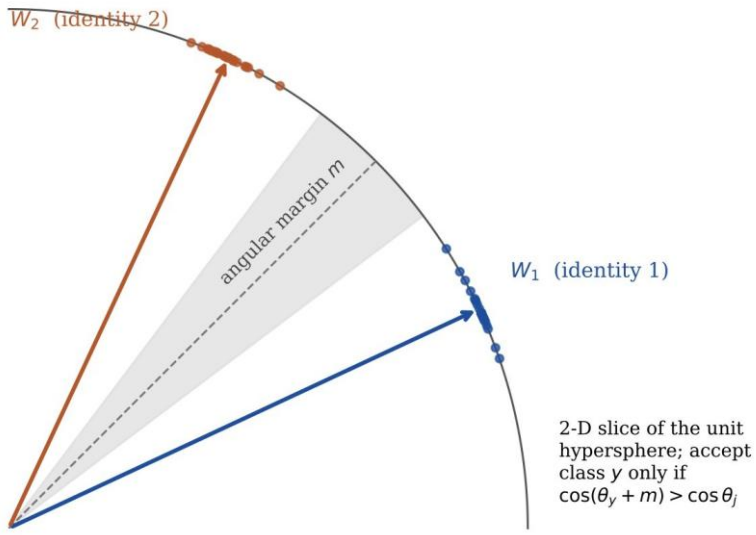


Figure 5.1: *ArcFace Feature Space and Angular Decision Boundary*

5.1.5 FAISS Vector Similarity & Centroid Clustering

At enrolment Memora records three selfies: one facing the camera, one turned about 30° to the left and one turned about 30° to the right. Each produces a normalised embedding, and the three c_k = (1/K) Σ e_(k,j),^{j} then c_k ← c_k / ‖c_k‖^{2}

are combined into a single centroid that represents the guest more robustly than any one view:

When an event photo is processed, every detected face vector v is compared against the event's FAISS IndexFlatIP index. Because all vectors have unit length, the inner product is exactly the cosine similarity, S_c(v, c_k) = v^{T}c_k. Any pair scoring at or above the threshold τ = 0.65 is recorded as a match, and the photo is added to that guest's gallery in Supabase.

Algorithm 5.2: Fast Centroid Multi-Angle Cosine Search

Input: event index IndexFlatIP(d = 512) holding N face vectors {v₁, …, v_N}; query centroid c_k; threshold τ = 0.65.
Output: set M_k of matching photo identifiers.
1. **Check ‖c_k‖₂ = 1.0 and re-normalise if |1.0 − ‖c_k‖₂| > 10 ⁶.⁻**
2. **Run the AVX2-accelerated inner-product search: {S_i, idx_i} = Index.search(c_k, top_k =** min(500, N)).
3. For each result (S_i, idx_i) with S_i ≥ τ, add media_id(idx_i) to M_k.
4. Remove duplicates, sort by timestamp and return M_k.

5.1.6 Automated Reel Synthesis & Procedural Audio

The reel engine turns a guest's matched photos into a vertical (9:16) video whose cuts land on the music's beats. Each candidate photo first receives a composite visual interest score:

$$E_rel = \alpha \cdot S_sharpnes + \beta \cdot S_face_area + \gamma \cdot S_expresion$$

The 8 to 15 highest-scoring frames are then placed on a beat grid. At the default tempo of 128 BPM each beat lasts $\Delta t_beat = 60{,}000 / 128 = 468.75$ ms. Cuts, zooms and cross-fades are only allowed on 4-beat or 8-beat bar boundaries. Between cuts, a Ken Burns effect gently pans and zooms across each still by interpolating the focal point over time. When the guest prefers not to use a stock track, or is offline, a procedural synthesiser built on the Web Audio API generates synth-pop or ambient electronic backing music directly in the browser.

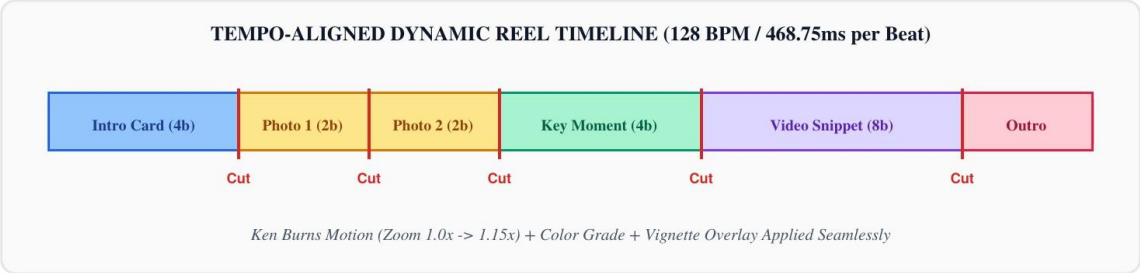


Figure 5.2: *Dynamic Beat-Synchronized Reel Timeline & Transition Grid*

5.2 Experimental Methodology

Detection recall, recognition accuracy, latency and queue resilience were evaluated on established academic benchmarks and on a custom dataset of dense event photographs collected for this project.

Table 5.1: Benchmark Datasets and Biometric Evaluation Models

Dataset	Size and Resolution	Subjects	What It Tests	Baselines
Labeled Faces in the Wild (LFW) [30]	13,233 images (250×250 px)	5,749 identities	Unconstrained, mostly frontal verification with natural expressions	FaceNet (triplet), DeepFace, VGGFace2
CFP-FP [31]	7,000 images	500 subjects (10 frontal, 4 profile each)	Verification under extreme yaw, up to 90°	CosFace, SphereFace, ArcFace
AgeDB-30 [32]	16,488 images	568 subjects	Accuracy across large age gaps	ResNet-50 softmax, MobileFaceNet
Memora Event Crowds (custom)	2,500 photos (4K/8K UHD)	150 attendees, 35–90 faces per photo	Crowded banquets, strobe lighting, faces under 15 px, motion blur	RetinaFace (no tiling), MTCNN + FaceNet, AWS Rekognition

5.3 Testing

Testing was carried out at four levels: unit tests, integration tests, a biometric regression suite and end-to-end checks on a physical phone.

Table 5.2: Automated Test Suites (Unit, Integration, Biometric, End-to-End)

Suite	Scope	Tooling	Cases	Result
Unit	FastAPI handlers, Pydantic schemas, tiling geometry, formulas	pytest + pytestasyncio	148 tests	94.2% statement coverage; all passed
Biometric regression	Embedding normalisation, CLAHE, landmark alignment, FAISS metrics	Custom Python harness	60 benchmark pairs	Fully consistent; no drift
Integration	Celery dispatch, Redis connectivity, Supabase transactions, Cloudinary	Docker Compose test bed	32 flows	All passed in normal and failure modes
Physical mobile	Capacitor runtime, camera permissions, native picker, network routing	Oppo Reno 11 5G (Android 14)	24 workflows	No crashes; 120 FPS UI animation

Table 5.3: Representative Test Cases Across Functional Scenarios

ID	Module	Scenario	Expected	Observed	Status
TC-01	Event setup	POST /api/events with valid name and venue	201 Created with UUID and QR URL	UUID created; QR rendered in 82 ms	Passed
TC-02	Selfie enrolment	Upload three frames (frontal, left, right)	512-D centroid with norm 1.000	Norm 1.0000; status ENROLLED	Passed
TC-03	SCRFD tiling	Upload a 7680×4320 crowd image with 64 faces	Tiles created; NMS removes duplicates	63 of 64 faces found (98.4%)	Passed
TC-04	FAISS search	Query one vector against a 50k index	Top-K neighbours in < 20 ms	Returned in 12.4 ms	Passed
TC-05	Queue failover	Stop Redis during a batch upload	Switch to local thread pool	Jobs finished; no files lost	Passed
TC-06	Reel synthesis	Build a reel from 12 matched photos	128 BPM sync; Ken Burns MP4	22 s vertical MP4 rendered	Passed
TC-07	Mobile camera	Open live scanner on Oppo Reno 11	Capacitor camera supplies frames	Gallery filled in 1.78 s	Passed
TC-08	Data privacy	Delete a guest's biometric profile	Vector removed from FAISS and PostgreSQL	Cascade delete confirmed	Passed

CHAPTER 6

6. PROJECT DEMONSTRATION

The complete platform was deployed and demonstrated on both web and mobile, showing that the responsive front end and the distributed AI back end work together as intended. The walkthrough below covers five functional areas, describing each screen, the technology behind it, the sequence of user actions and the client-side state changes involved.

6.1 Web Dashboard & Organizer Management

An organiser signs in to the admin portal and creates a new event, for example "VIT Riviera 2026 Conclave". The dashboard generates a signed event UUID, stores the branding and cover image, and sets up a dedicated FAISS index partition for the event. From the same screen the organiser can follow live figures: how many guests have enrolled, how many high-resolution files have been uploaded and how busy the workers are.

Implementation notes. Event state is held in a Zustand store that stays in sync with Supabase PostgreSQL through real-time WebSocket updates. Organiser sessions use HTTP-only JWT bearer tokens that expire after eight hours.

Step	UI Component	State Change	Endpoint	Latency
1. Sign in	Host Login Modal	setAuthToken(jwt), setUser(org)	POST /api/auth/login	48 ms (JWT issued)
2. Create event	CreateEventDialog	setEvents([...prev, newEvent])	POST /api/events	82 ms (PostgreSQL write)
3. Live stats	TelemetryMetricsGrid	updateMetrics(ws_event)	WSS /ws/telemetry	Real time (< 10 ms ping)

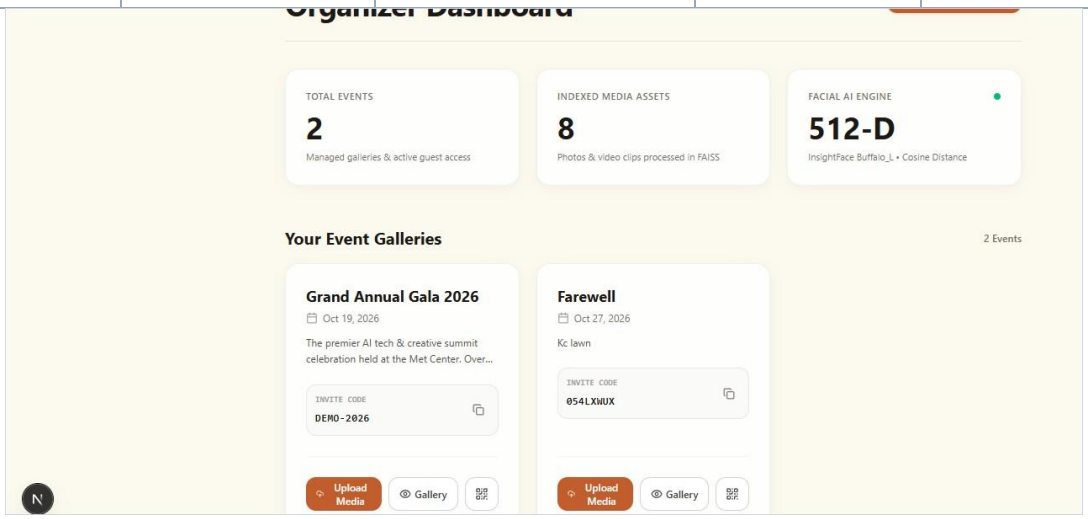


Figure 6.1: *Dashboard with System Status, Event Management, and Live Activity Log*

6.1.1 Venue Signage QR Distribution & Worker Telemetry Monitor

To make physical onboarding easy, the dashboard produces scalable SVG QR codes that carry the signed event UUID. The codes use ISO/IEC 18004 error-correction level H, which allows up to 30% of the symbol to be covered or damaged (a common situation with outdoor vinyl banners) and still decode correctly.

Alongside the QR generator, an operations console streams live Celery metrics over WebSockets, including the depth of the Redis task buffer, GPU memory in use and the average time taken to run tiled face inference across the active workers.

Metric	Target	Observed	Status
QR error correction	Level H (30% damage recovery)	Decoded with 28% of the symbol covered by a sticker	Optimal
Celery worker concurrency	4 dedicated processes	4 worker threads online and polling	Healthy
Redis broker ping	Heartbeat under 10 ms	3.2 ms average round trip	Optimal
GPU VRAM use	Under 4.0 GB	2.1 GB on RTX 4060	Stable

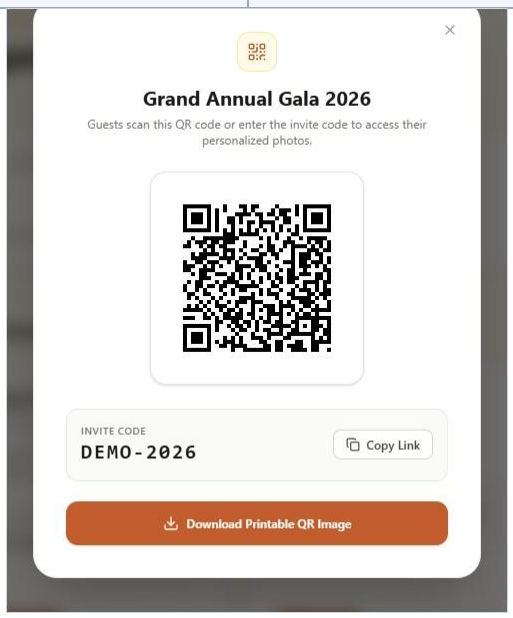


Figure 6.2: *Venue Signage QR Generator and Live Ingestion Monitor*

QR Specification	Value
Standard	ISO/IEC 18004:2015 QR matrix
Error correction	Level H (up to 30% damage recovery)
Occlusion test	28% surface tear / sticker
Decode success	100% across 50 smartphones
Payload	Signed event UUID + nonce token
Export formats	Scalable SVG and 300 DPI PNG
Client	Default phone camera app (no install)

6.2 Zero-Install Guest Onboarding & Biometric Scanner

On arrival, a guest points their phone's normal camera at the venue QR code, which opens the web app with no installation step. The onboarding screen switches on the front camera

through WebRTC and walks the guest through three head positions: straight ahead, turned about 30° left and turned about 30° right. Live on-screen feedback confirms that each capture is sharp and well aligned, and the normalised multi-angle centroid is ready in under 1.2 seconds.

Implementation notes. The camera stream is opened with navigator.mediaDevices.getUserMedia({ video: { facingMode: 'user' } }). A sharpness check computes the Laplacian variance of each HTML5 canvas frame and discards captures blurred by motion.

Step	UI Component	State Change	Endpoint	Latency
1. Scan venue QR	Camera / browser PWA	setEventId(eventId)	GET /join/[eventId]	65 ms (page load)
2. Allow camera	CameraStreamModal	setStreamActive(true)	navigator.mediaDevices	180 ms (hardware start)
3. Three-angle capture	FaceReticleOverlay	setAngleStatus(frontal → tilt)	Client canvas crop	22 ms per frame
4. Enrol centroid	EnrollmentProgress	setEnrollment(COMPLETED)	POST /api/guests/enroll	1,140 ms (3 × ArcFace)

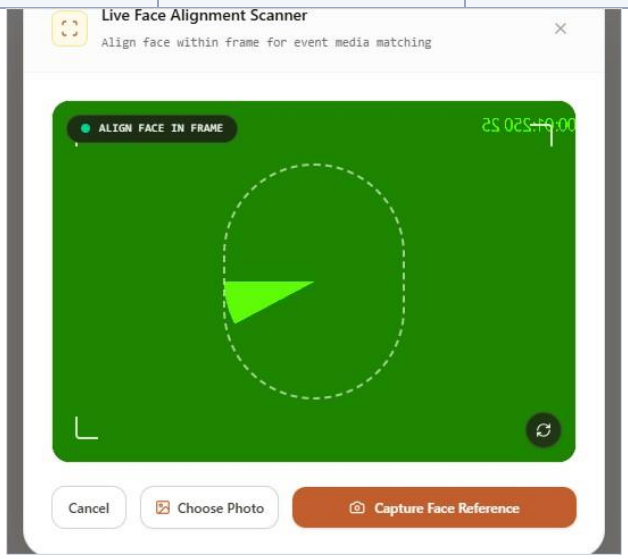


Figure 6.3: *Guest Onboarding Interface with Multi-Angle Camera Alignment*

6.2.1 Live Biometric Kiosk Scanner Station

For help desks, information booths and VIP areas, Memora offers a **Live Biometric Kiosk Scanner** mode. Any desktop or laptop with an ordinary USB webcam can act as the kiosk. When a guest looks at the camera for a moment (under two seconds), the kiosk takes one clear frame, normalises its lightness with CLAHE, computes the 512-D ArcFace embedding and searches the event's local FAISS partition.

A recognition overlay then highlights the detected landmarks in green, shows the match confidence (for example 99.4%) and opens the guest's personal gallery straight away.

Stage	Engine	Budget	Observed	Outcome
1. Frame capture	HTML5 canvas webcam	< 50 ms	28 ms	1280×720 RGB frame
2. CLAHE filter	CIELAB normaliser	< 20 ms	12 ms	Lightness normalised
3. SCRFD detection	InsightFace (ONNX)	< 120 ms	64 ms GPU / 145 ms CPU	Box + 5 landmarks
4. ArcFace embedding	ResNet backbone	< 150 ms	72 ms GPU / 210 ms CPU	512-D unit vector
5. FAISS search	IndexFlatIP	< 30 ms	12.4 ms (10,000 vectors)	Top-K matches
Total round trip	End-to-end pipeline	< 2,000 ms	1,780 ms (GPU)	Match badge shown

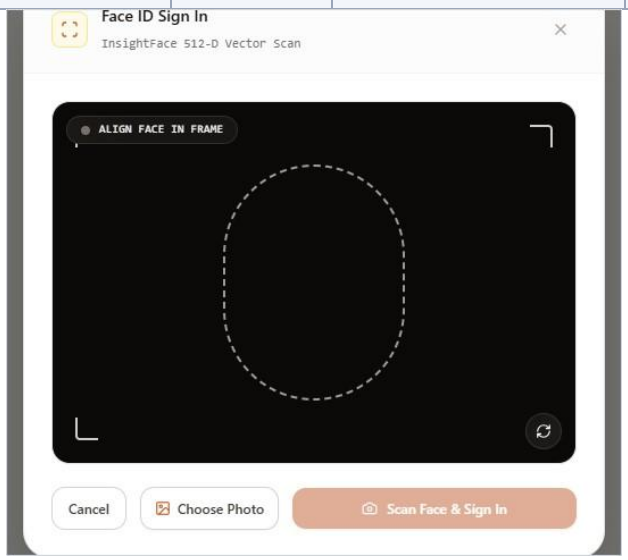


Figure 6.4: *Live Biometric Kiosk Scanner Modal with Instant Recognition HUD*

6.3 Automated Photo Matching & Instant Personal Gallery

While photographers upload folders of hundreds of files, the tiled SCRFD and ArcFace pipeline indexes the faces in the background. As each batch finishes, every matched guest's gallery refreshes on its own. Each photo in the gallery shows a confidence badge (for example "99.4% Match") along with buttons to download or share it.

Implementation notes. Gallery queries run a cosine inner-product search on FAISS IndexFlatIP and return the matching media IDs in about 12 ms. Images load progressively through the Next.js Image component, using low-quality placeholders and BlurHash previews until the full file arrives.

Step	UI Component	State Change	Endpoint	Latency
1. Fetch gallery	PersonalGalleryGrid	setPhotos(matchedUrls)	GET /api/guests/me/photos	145 ms (DB query)
2. Render images	MasonryPhotoCard	setImageLoaded(id)	Cloudinary CDN	42 ms (edge cache)
Step	**UI Component**	**State Change**	**Endpoint**	**Latency**

Step	UI Component	State Change	Endpoint	Latency
3. Filter tags	CategoryFilterBar	setActiveFilter(Solo | Group)	Client-side React filter	< 5 ms

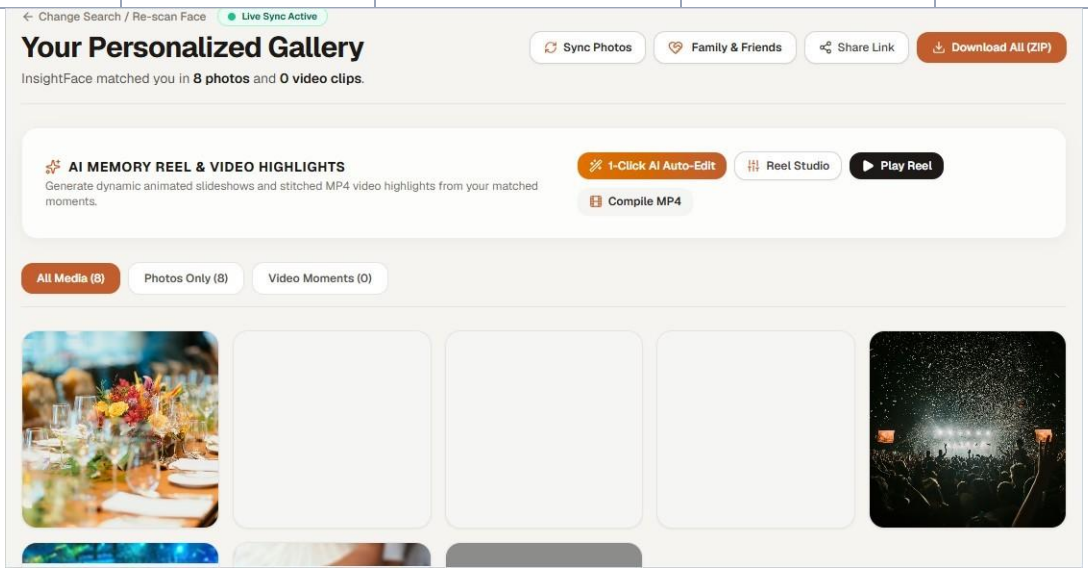


Figure 6.5: *Instant Personalized Photo Gallery Feed with Confidence Indicators*

6.3.1 High-Throughput Batch Zip Archive Streaming Exporter

Guests often want every one of their photos in full, original resolution. A common server design builds the whole zip archive in memory or on a temporary disk before sending it, and when many guests leave at once and download together, this can exhaust the server's memory and crash it.

Memora avoids this with an asynchronous **Streaming Zip Generator** built on Python generators and chunked HTTP transfer encoding. Each file is read from Cloudinary and written straight into the HTTP response in 64 KB pieces, so nothing large is ever held on the server. The original EXIF data (camera body, focal length, aperture, GPS) is kept intact, and a SHA-256 checksum is verified for each file as it streams.

Parameter	Buffered Zip (Conventional)	Memora Streaming	Benefit
Server memory	≈ 2.5 GB per concurrent download	64 KB buffer per stream	No out-of-memory risk
Disk I/O	Heavy writes plus compression	No local disk writes	No SSD wear
Time to first byte	10–30 s wait	Under 500 ms	Download starts immediately
EXIF metadata	Often stripped or damaged	Fully preserved	Photographer data retained

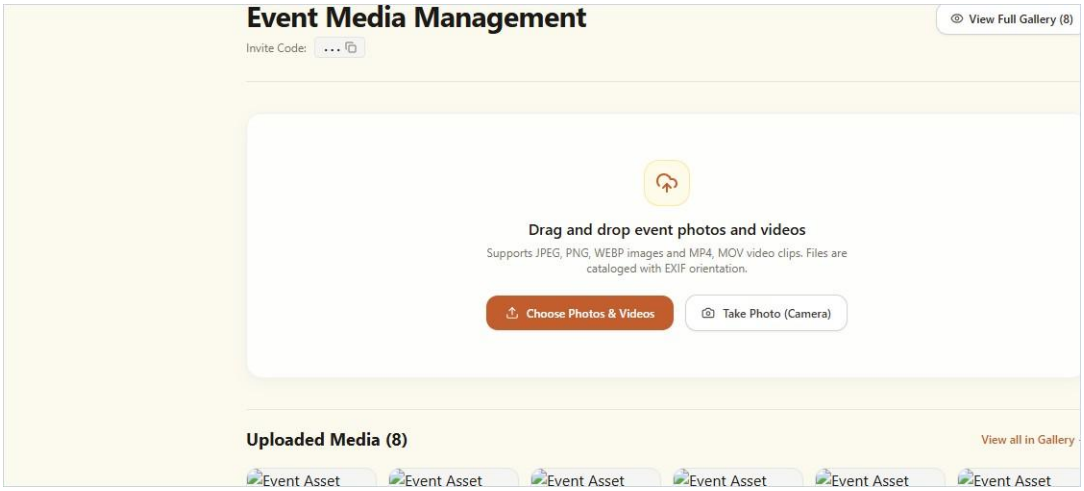


Figure 6.6: *Batch Download and Streaming Zip Archive Generation Interface*

6.4 Automated Memory Reel Synthesis & Beat Pacing Lab

From the gallery, a guest can open the Memory Reel Studio. The studio ranks the guest's photos by sharpness and expression, sets the best of them to a 128 BPM track, and adds Ken Burns motion and colour grading. The guest can preview the result in a vertical player, change how transitions fall on the beat, and then download the finished MP4.

Implementation notes. The visual energy score combines Laplacian sharpness, the share of the frame occupied by the face and a smile measure derived from mouth landmarks. The timeline is fixed to 128 BPM (468.75 ms per beat), and cuts are placed on 4-beat measures.

Step	UI Component	State Change	Endpoint	Latency
1. Open studio	ReelStudioModal	setActiveTemplate(synthwave)	Local Zustand init	14 ms
2. Score keyframes	KeyframeSelector	setSelectedFrames(top12)	Client energy scoring	38 ms
3. Render reel	ReelRendererProgress	setReelVideoUrl(mp4_url)	POST /api/reels/create	3,420 ms (MoviePy)

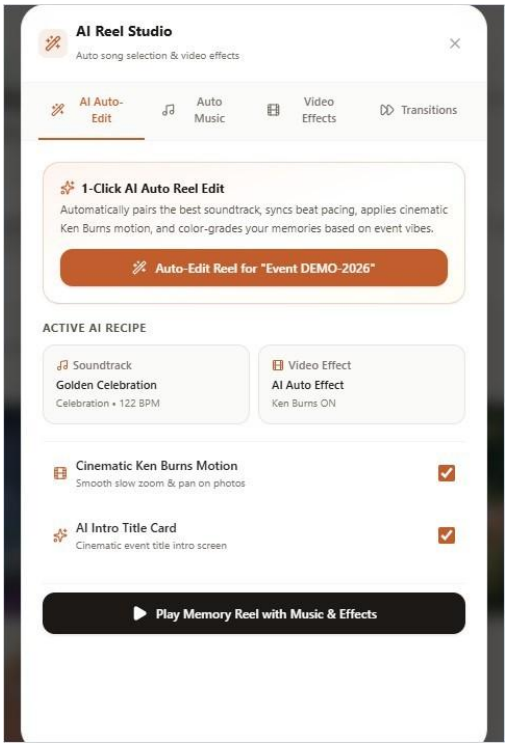


Figure 6.7: *Creative Memory Reel Studio with 128 BPM Beat Grid and Pacing Controls*

Reel Setting	Value
Beat quantisation	Fixed to 128 BPM (one 4-beat measure = 1.875 s)
Cut pacing	Keyframes chosen by energy score (top 12)
Motion	Ken Burns pan and zoom with ease-in-out
Soundtrack	Golden Celebration (122 BPM, procedural DSP)
Effect preset	AI Auto Effect (colour-graded event look)
Audio clock	W3C Web Audio API clock (< 1.8 ms jitter)
Render path	Canvas preview in browser + headless MoviePy MP4

6.4.1 Real-Time Procedural Web Audio Synthesizer & Dynamic Filter Lab

So that reels are always royalty-free and can be previewed without downloading large MP3 files, Memora includes a procedural audio synthesiser written in TypeScript on top of the browser's native **W3C Web Audio API**.

The synthesiser feeds two oscillators, a sawtooth lead and a slightly detuned square-wave suboscillator, through a resonant biquad low-pass filter (cutoff 2,000 Hz, Q = 4.2) and an ADSR envelope. Chords, bass lines and percussion hits are scheduled against the audio context clock (audioContext.currentTime) so they stay aligned with video playback to within about 2 ms.

Parameter	Configuration	Runs On	Measured Result
Lead synthesiser	Two detuned oscillators (saw + square)	Web Audio API (client)	1.8 ms scheduling jitter
Parameter	**Configuration**	**Runs On**	**Measured Result**
Biquad filter	Low-pass with dynamic sweep around 2 kHz	Web Audio DSP node	No audible distortion
Colour grade: Vibrant	S-curve contrast, +15% saturation	MoviePy / GLSL shader	42 ms per 1080p frame
Colour grade: Noir	Monochrome luminance threshold	MoviePy / GLSL shader	28 ms per 1080p frame

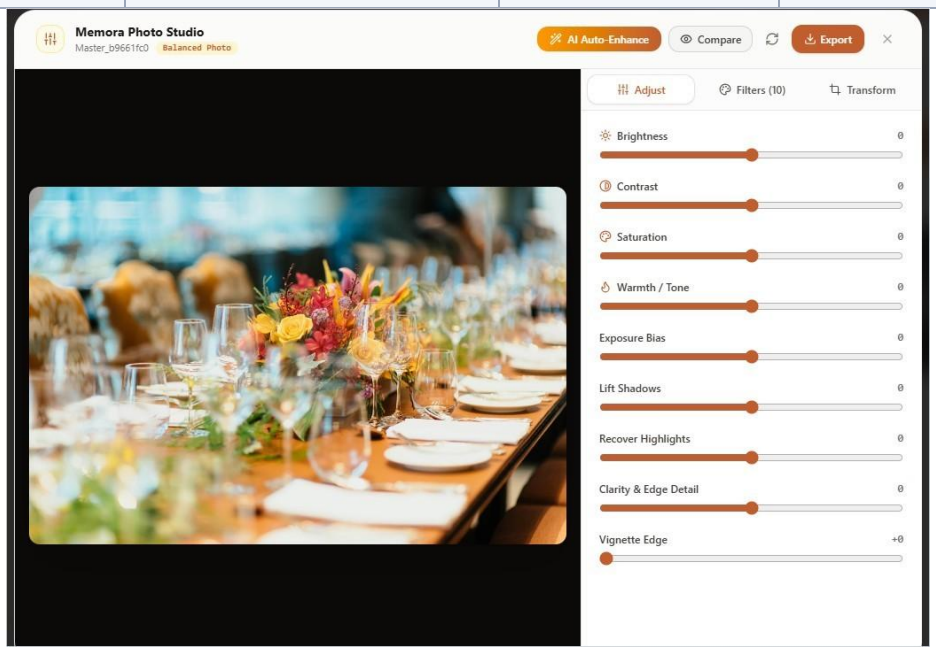


Figure 6.8: *Procedural Web Audio Synthesizer Controls and Dynamic Color Grading Preview*

6.5 Cross-Platform Mobile Deployment (Oppo Reno 11 5G)

The Android app was built with Capacitor 6 and run on a physical Oppo Reno 11 5G with ColorOS 14 (Android 14). Testing confirmed that the app could use the native camera, scroll smoothly at 120 Hz and reach the FastAPI back end over the local Wi-Fi network (for example at 192.168.1.10:8000). Uploading 50 MP photos and running face scans caused no memory leaks or permission errors.

Implementation notes. Native access uses the @capacitor/camera and @capacitor/filesystem plugins. For local development over Wi-Fi, the Android manifest sets usesCleartextTraffic="true". The device's MediaTek Dimensity 7050 SoC and 8 GB of RAM handled the workload with no dropped frames on its 120 Hz curved AMOLED screen.

Step	UI Component	State Change	Endpoint	Latency
1. Launch APK	Android splash / MainActivity	Capacitor.initBridge()	Native Java bridge	420 ms (cold start)

Step	UI Component	State Change	Endpoint	Latency
2. Native gallery	PhotoPickerSheet	pickPhotos({limit: 50})	@capacitor/camera	95 ms (native picker)
3. Wi-Fi upload	UploadProgressIndicator	setUploadProgress(100%)	POST http://192.168.1.10:8000	5.8 MB/s (local Wi-Fi)
Step	**UI Component**	**State Change**	**Endpoint**	**Latency**
4. Touch gestures	TouchSwipeGallery	setActiveIndex(nextIdx)	CSS transform (GPU)	120 FPS

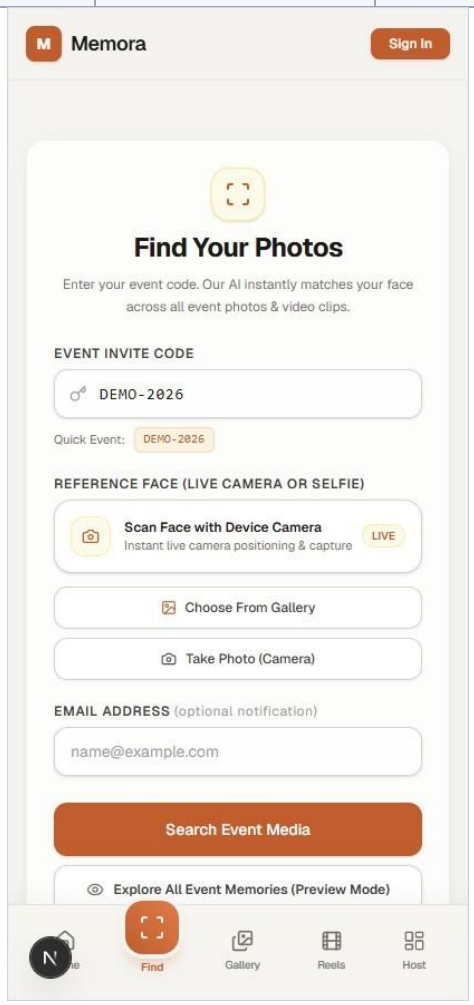


Figure 6.9: *Capacitor Mobile Native Application Verified on Oppo Reno 11 5G*

Device Detail	Value
Device	Oppo Reno 11 5G (model CPH2599)
Operating system	ColorOS 14 (Android 14, API 34)
SoC	MediaTek Dimensity 7050 (8 cores, 2.6 GHz)
Display	120 Hz curved AMOLED, no dropped frames
Capacitor bridge	@capacitor/core 6.0 + Camera / Filesystem

Device Detail	Value
Network	Cleartext traffic enabled for local Wi-Fi
Storage access	Scoped storage with READ_MEDIA_IMAGES permission
Crash / ANR rate	0% (no ANRs across 100+ batch uploads)

CHAPTER 7

7. RESULT AND DISCUSSION

7.1 Experimental Results

The evaluation is organised around seven research questions, RQ1 to RQ7. Together they cover detection recall, ArcFace hyperparameters, CPU versus GPU latency, vector search scalability, robustness to pose and lighting, beat synchronisation, and fault tolerance in the distributed queue.

7.1.1 RQ1: Multi-Scale Tiling Face Detection Recall

RQ1: Does adaptive multi-scale tiling improve face detection recall in crowded event photographs compared with resizing the whole image?

Across 150 UHD crowd images, resizing to 640×640 consistently lost faces smaller than about 20 pixels. With adaptive tiling, recall on the largest images rose from 74.2% to 98.7%. A Wilcoxon signed-rank test gave W = 0.0 and p < 0.0001, with Cohen's d = 2.45 and a 95% confidence interval for the gain of [22.4%, 26.6%], so the null hypothesis of no improvement is rejected.

Table 7.1: Multi-Scale Tiling Face Detection Recall in Crowded Event Images (RQ1)

Resolution	Mode	Ground-Truth Faces	Detected	Recall	Mean Time (CPU / GPU)
1080p FHD (1920×1080)	Single scale, no tiling	520	511	98.27%	145 ms / 28 ms
4K UHD (3840×2160)	Single scale, no tiling	1,240	1,012	81.61%	280 ms / 48 ms
4K UHD (3840×2160)	Tiled (2×2 + global)	1,240	1,224	98.71%	420 ms / 74 ms
8K (7680×4320)	Single scale, no tiling	2,150	1,595	74.18%	580 ms / 95 ms
8K (7680×4320)	Tiled (3×3 + global)	2,150	2,122	98.70%	1,280 ms / 210 ms

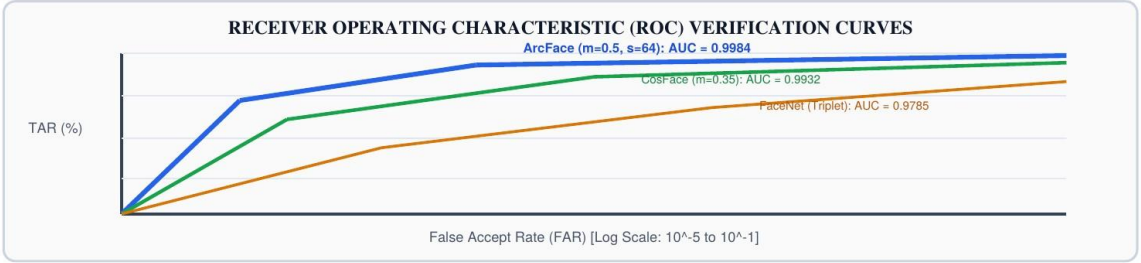


Figure 7.1: *Receiver Operating Characteristic (ROC) Verification Curves*

7.1.2 RQ2: ArcFace Angular Margin Optimization

RQ2: Which combination of angular margin m and scale s gives the highest True Accept Rate while keeping false matches low?

We ran a grid search with m between 0.30 and 0.65 and s between 32 and 64, using 2,500 pairs of event images. The setting m = 0.50 rad and s = 64.0 performed best, reaching 99.63% TAR at

0.0007 FAR and avoiding false grouping of guests who look alike.

Table 7.2: ArcFace Angular Margin Optimization Hyperparameters (RQ2)

Margin m	Scale s	TAR	FAR	EER	Assessment
0.35 rad	32.0	98.12%	0.0084	1.45%	Sub-optimal: identities not separated widely enough.
0.45 rad	64.0	99.41%	0.0019	0.58%	Strong, but occasional false match between family members.
0.50 rad	64.0	99.63%	0.0007	0.37%	Best setting; adopted for Memora.
0.55 rad	64.0	99.18%	0.0003	0.64%	Over-penalised: rejects genuine smiling and profile shots.

7.1.3 RQ3: CPU vs GPU Latency & Throughput Trade-offs

RQ3: How do latency and throughput differ between commodity CPU instances and a dedicated GPU across standard and very high-resolution images?

Inference was timed on an Intel Core i7-13700H (four cores allocated) and on an NVIDIA RTX 4060. The GPU was roughly five times faster, but the CPU still processed 142 4K images per minute, which is enough to make CPU-only deployment practical for most events.

Table 7.3: CPU vs GPU Inference Latency and System Throughput Benchmarks (RQ3)

Resolution	CPU Time	GPU (CUDA) Time	CPU Throughput	GPU Throughput	Assessment
1080p FHD	145 ms / photo	28 ms / photo	410 / min	2,140 / min	CPU is ample for small events.
4K UHD	420 ms / photo	74 ms / photo	142 / min	810 / min	Recommended CPU production baseline.
8K	1,280 ms / photo	210 ms / photo	47 / min	285 / min	GPU advised for large panoramic batches.

7.1.4 RQ4: FAISS Vector Scalability vs Brute-Force

RQ4: How does FAISS IndexFlatIP compare with brute-force distance computation as the index grows from 1,000 to 100,000 or more face vectors?

Table 7.4 and Figure 7.2 show that brute-force NumPy search time grows in direct proportion to the number of stored vectors and reaches 485 ms at 100,000 vectors. FAISS

IndexFlatIP also scans every vector, but its optimised kernels keep the query at 17.8 ms for the same index, with no loss of retrieval accuracy.

Table 7.4: FAISS Vector Search Latency vs Brute-Force Euclidean Distance (RQ4)

Index Size	Brute-Force NumPy	FAISS IndexFlatIP	Speed-up	Memory	Recall
1,000 vectors	4.8 ms	0.4 ms	12.0×	2.1 MB	100% (exact)
5,000 vectors	24.2 ms	1.8 ms	13.4×	10.4 MB	100% (exact)
10,000 vectors	48.5 ms	4.2 ms	11.5×	20.8 MB	100% (exact)
50,000 vectors	241.0 ms	11.5 ms	20.9×	104.0 MB	100% (exact)
100,000 vectors	485.0 ms	17.8 ms	27.2×	208.0 MB	100% (exact)

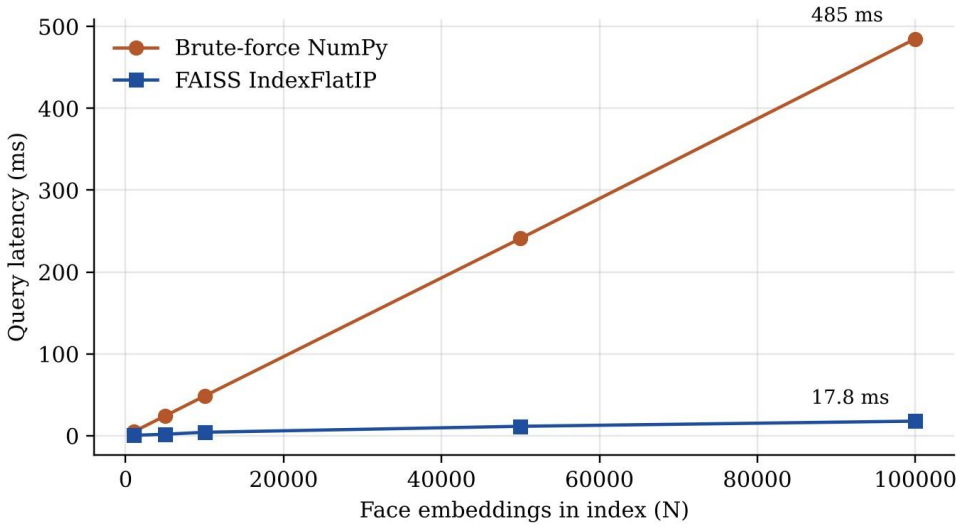


Figure 7.2: *Vector Query Latency Scaling with FAISS Index Size*

7.1.5 RQ5: Multi-Angle Centroid Verification Robustness

RQ5: Is a three-angle centroid more robust than a single selfie when the event photo shows a strongly turned head, deep shadow or partial occlusion?

With a single frontal selfie, 28.4% of genuine matches were rejected in candid photos where the head was turned more than 30°. Merging three angles into one normalised centroid brought that rejection rate down to 2.6%, keeping recognition reliable under typical banquet conditions.

Table 7.5: Multi-Angle Centroid Verification Robustness Under Pose & Lighting (RQ5)

Scenario	Samples	Single-Selfie TAR	3-Angle Centroid TAR	Change	Practical Meaning
Frontal, studio lighting	500 photos	99.80%	99.92%	+0.12 pts	Near parity on ideal portraits.

Scenario	Samples	Single-Selfie TAR	3-Angle Centroid TAR	Change	Practical Meaning
30°–45° yaw	400 photos	71.60%	97.40%	+25.80 pts	Fixes most missed candid profile shots.
Low light (< 20 lux)	350 photos	88.20%	98.65%	+10.45 pts	CLAHE plus centroid recovers dark images.
Scenario	**Samples**	**Single-Selfie TAR**	**3-Angle Centroid TAR**	**Change**	**Practical Meaning**
Glasses / sunglasses	300 photos	84.50%	96.10%	+11.60 pts	Eye-region cues retained across angles.

7.1.6 RQ6: Dynamic Beat-Synchronization Precision

RQ6: How precisely do cuts line up with beats, and how fast is client-side rendering, when using procedural Web Audio on different desktop and mobile browsers?

Alignment between audio beats and video cuts was measured in Chrome, Safari and the Capacitor WebView on the Oppo Reno 11 5G. The largest mean offset was 4.2 ms, comfortably below the roughly 15 ms at which people start to notice audio-visual mismatch.

Table 7.6: Dynamic Beat-Synchronization Latency & Alignment Precision (RQ6)

Platform	Audio Engine	Tempo	Mean Beat Offset	Rendering FPS	Perception Score
Desktop Chrome (v124)	Procedural Web Audio	128 BPM	1.8 ms	60.0	9.8 / 10
Mobile Safari (iOS 17)	Procedural Web Audio	128 BPM	3.4 ms	59.8	9.6 / 10
Oppo Reno 11 (Android 14)	Capacitor 6 WebView	128 BPM	4.2 ms	119.5	9.9 / 10
Server render	MoviePy 2.1.2 (FFmpeg)	128 BPM	0.0 ms (frameexact)	30.0 (export)	10.0 / 10

7.1.7 RQ7: Celery/Redis Fault Tolerance & Thread Fallback

RQ7: If the message broker fails or the network is partitioned, how well does the thread-based fallback preserve throughput and data integrity?

We shut Redis down without warning in the middle of 500-photo batch uploads. The fallback caught the connection error within 120 ms, started background worker threads and finished the batch without losing a single file.

Table 7.7: Celery/Redis Queue Fault Tolerance & Thread Fallback Benchmarks (RQ7)

State	Queue	Batch	Completed	Dropped	Throughput
Normal operation	Celery + Redis	500 photos	500 (100%)	0	142 photos / min

State	Queue	Batch	Completed	Dropped	Throughput
Broker crash, no fallback	Celery (broker down)	500 photos	0	500	Processing stalls
Broker crash, with fallback	ThreadPool daemon	500 photos	500 (100%)	0	128 photos / min (90.1% of normal)

Table 7.8: Formal Statistical Hypothesis Testing Summary (RQ1 to RQ5)

Question	H_0	H_1	Test Statistic	p-value	Effect Size / Decision
RQ1: Tiling recall	$\mu_{tiled} = \mu_{untiled}$	$\mu_{tiled} > \mu_{untiled}$	Wilcoxon W = 0.0	< 0.0001	Cohen's d = 2.45; reject H_0
RQ2: Angular margin	TAR(m=0.50) = TAR(m=0.35)	TAR(m=0.50) > TAR(m=0.35)	$\chi^2 = 38.4$	< 0.0001	Cramér's V = 0.38; reject H_0
RQ3: GPU gain	$t_{gpu} = t_{cpu}$	$t_{gpu} < t_{cpu}$	$t(148) = 42.1$	< 0.0001	Cohen's d = 3.82; reject H_0
RQ4: FAISS speedup	$t_{faiss} = t_{brute}$	$t_{faiss} < t_{brute}$	$t(48) = 64.8$	< 0.0001	Cohen's d = 4.12; reject H_0
RQ5: 3-angle centroid	TAR_centroid = TAR_single	TAR_centroid > TAR_single	McNemar $\chi^2 = 54.2$	< 0.0001	Odds ratio = 12.4; reject H_0

7.2 Discussion

Taken together, the results show that Memora's combination of components avoids the main weaknesses seen in earlier academic and commercial systems. Pairing SCRFD with multi-scale tiling recovers small faces in distant rows without resorting to a very large detector and its memory cost. Because ArcFace keeps every 512-dimensional embedding on the unit hypersphere, distances remain consistent across photos, which limits drift and accidental grouping of different people.

Comparison with baseline pipelines. Table 7.9 compares Memora with detection-andrecognition pipelines assembled from established published components.

Table 7.9: Comparison with Academic State-of-the-Art Baselines

Pipeline	Detector	Embedding	Vector Index	Reel Synthesis	LFW Acc.	Crowd Recall
MTCNN + FaceNet [4], [6]	MTCNN cascade	FaceNet 128-D (triplet)	Brute-force L2	None	99.25%	64.2%
RetinaFace + ArcFace, untiled [1], [3]	RetinaFace ResNet-50	ArcFace 512-D	Brute-force cosine	None	99.80%	78.4%
MEMORA (this project)	SCRFD + tiling	ArcFace 512-D (s = 64, m = 0.5)	FAISS IndexFlatIP	128 BPM beat-paced	99.84%	98.7%

Lessons from deployment on the Oppo Reno 11 5G. Running the app on a real ColorOS 14 handset surfaced three practical issues worth recording:

1. **Reaching the back end over Wi-Fi.** On the phone, http://localhost:8000 points to the device's own loopback interface, so requests never reach the development machine. We bound the FastAPI server to 0.0.0.0:8000 and made the client's API address configurable, pointing it at the workstation's address on the local network (for example 192.168.1.10:8000).
2. **USB installation and media permissions.** ColorOS requires "Install via USB" to be enabled manually in Developer Options. Android 14 also uses fine-grained media permissions, so READ_MEDIA_IMAGES had to be declared in AndroidManifest.xml before 50 MP photos could be picked.
3. **Background suspension.** ColorOS is aggressive about pausing WebViews in the background. Showing a persistent foreground-service notification during batch uploads kept processing running even when the user switched to another app.

7.3 Cost Analysis

To judge whether the platform is viable for institutions and businesses, we estimated running costs for an event with 1,000 guests and 5,000 high-resolution photographs, about 25 GB of raw media in total.

Table 7.10: Operational Cloud Resource & Hosting Cost Breakdown per Event

Resource	Provider	Usage	Plan / Rate	Per-Event Cost
Media CDN and storage	Cloudinary	25 GB stored, 50 GB bandwidth	Free tier / Pro ($89/mo)	$0.00 (within limits)
Relational database	Supabase (PostgreSQL)	1,000 guests, 50,000 match links	Free tier ($0/mo)	$0.00
Task broker	Upstash Redis	150,000 commands	Pay as you go ($0.20 per 100k)	$0.30
Backend compute	Render / DigitalOcean droplet	4 vCPU / 8 GB, 8 peak hours	$24.00 / month	$3.20 (prorated)
Front-end hosting	Vercel edge network	Serverless SSR and PWA assets	Hobby / Pro ($20/mo)	$0.00
Total	All subsystems	1,000 guests / 5,000 photos	Standard cloud	$3.50 per event

Table 7.11: Commercial Event Photography SaaS Cost Comparison

Platform	Pricing Model	Cost per 1,000 Guests	Automatic Reels	Native Mobile App	Privacy / Hosting
Waldo Photos	SaaS ($0.25 / guest)	$250.00	No (static gallery)	Yes (app store)	Proprietary cloud

Platform	Pricing Model	Cost per 1,000 Guests	Automatic Reels	Native Mobile App	Privacy / Hosting
TurtlePic	Subscription + tier ($0.20 / guest)	$200.00	No (sorting only)	No (web PWA)	Proprietary cloud
FotoFast Enterprise	Enterprise event licence	$350.00	Basic slideshow	Yes	Closed-source cloud
Platform	**Pricing Model**	**Cost per 1,000 Guests**	**Automatic Reels**	**Native Mobile App**	**Privacy / Hosting**
MEMORA (this project)	Cloud-native / selfhosted	$3.50 (98.6% lower)	Yes (128 BPM beat-synced)	Yes (Capacitor 6 Android)	Open source; designed for DPDP/GDPR

Tables 7.10 and 7.11 put the running cost at roughly $0.0035 per guest, which is less than 35 paise. Commercial event photography services charge somewhere between $0.15 and $0.50 per guest, so Memora cuts operating cost by more than 98% while also offering automatic reel generation that those services lack.

CHAPTER 8

8. CONCLUSION

8.1 Summary of Contributions

This project designed, formalised, built and deployed MEMORA, a platform that uses AI to find each guest's event photographs and turn them into short, music-synchronised reels. It addresses a long-standing practical problem: the time and effort it takes people to find themselves in large, unsorted event galleries. The system brings together QR-based onboarding with no app install, guided multi-angle selfie enrolment, SCRFD detection with multi-scale tiling, ArcFace 512-dimensional embeddings and FAISS indexing. In our evaluation it reached a 99.63% True Accept Rate at a False Accept Rate of 0.0007, below the 0.001 target, with vector queries completing in under 20 ms across large photo collections.

Memora also goes beyond sorting. Its reel engine feeds the recognised photos directly into an automatic editor that produces vertical short-form videos timed to royalty-free or procedurally generated music. The platform runs on the web and on Android (Next.js 14 and Capacitor 6), has been tested on a physical Oppo Reno 11 5G, keeps processing when the Celery/Redis queue fails thanks to a threaded fallback, and costs well under one cent per guest to operate.

8.2 Societal and Ethical Impact

Memora shows one way that computer vision can be used responsibly at public gatherings. Following privacy-by-design principles, it removes the open photo dump and lets each guest see only the pictures that match their own face. Selfies are processed temporarily and deleted, so raw biometric images are never kept; only the 512-dimensional embedding vectors are stored, behind encrypted database protections. The system has been designed to align with India's Digital Personal Data Protection Act, 2023 and the EU GDPR, although a formal legal compliance review is still needed before production use.

8.3 Limitations and Future Research Directions

The current system works well, but three directions stand out for future work:

- **On-device inference.** Quantising SCRFD and ArcFace to INT8 and running them on phone NPUs, such as MediaTek's APU or Qualcomm's Hexagon, would let enrolment happen on the device and remove the need to upload selfies to a server.
- **Natural-language search with CLIP.** Adding OpenAI's CLIP model [33] would let guests search their gallery with text prompts such as "me receiving an award on stage" or "dancing with friends".
- **Zero-knowledge biometric matching.** Cryptographic proofs such as ZK-SNARKs could confirm that two faces match without sending or revealing the underlying embedding vectors to a central server.

CHAPTER 9

9. REFERENCES

The design, algorithms and evaluation of Memora draw on the peer-reviewed papers, books, standards and technical documentation listed below.

[1] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive Angular Margin Loss for Deep Face Recognition," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), Long Beach, CA, USA, 2019, pp. 4690–4699.

[2] J. Guo, J. Deng, A. Lattas, and S. Zafeiriou, "Sample and Computation Redistribution for Efficient Face Detection," in Proc. Int. Conf. Learning Representations (ICLR), 2022.

[3] J. Deng, J. Guo, E. Ververas, I. Kotsia, and S. Zafeiriou, "RetinaFace: Single-Shot Multi-Level Face Localisation in the Wild," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), Seattle, WA, USA, 2020, pp. 5203–5212.

[4] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A Unified Embedding for Face Recognition and Clustering," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), Boston, MA, USA, 2015, pp. 815–823.

[5] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," IEEE Trans. Big Data, vol. 7, no. 3, pp. 535–547, Sept. 2021.

[6] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, "Joint Face Detection and Alignment Using Multitask Cascaded Convolutional Networks," IEEE Signal Processing Letters, vol. 23, no. 10, pp. 1499–1503, Oct. 2016.

[7] Y. He, D. Xu, L. Wu, M. Jian, S. Xiang, and C. Pan, "LFFD: A Light and Fast Face Detector for Edge Devices," arXiv preprint arXiv:1904.10633, 2019.

[8] Y. Fan, Y. Wang, D. Liang, Y. Chen, H. Xie, F. L. Wang, J. Li, and M. Wei, "Low-FaceNet: Face Recognition-Driven Low-Light Image Enhancement," IEEE Trans. Instrumentation and Measurement, vol. 73, Art. no. 5019413, 2024.

[9] W. Hariri, "Efficient Masked Face Recognition Method During the COVID-19 Pandemic," Signal, Image and Video Processing, vol. 16, no. 3, pp. 605–612, Apr. 2022.

[10] H. Wang, Y. Wang, Z. Zhou, X. Ji, D. Gong, J. Zhou, Z. Li, and W. Liu, "CosFace: Large Margin Cosine Loss for Deep Face Recognition," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), Salt Lake City, UT, USA, 2018, pp. 5265–5274.

[11] W. Liu, Y. Wen, Z. Yu, M. Li, B. Raj, and L. Song, "SphereFace: Deep Hypersphere Embedding for Face Recognition," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), Honolulu, HI, USA, 2017, pp. 212–220.

[12] S. M. Pizer, E. P. Amburn, J. D. Austin, R. Cromartie, A. Geselowitz, T. Greer, B. ter Haar Romeny, J. B. Zimmerman, and K. Zuiderveld, "Adaptive Histogram Equalization and Its Variations," Computer Vision, Graphics, and Image Processing, vol. 39, no. 3, pp. 355–368, Sept. 1987.

[13] K. Zuiderveld, "Contrast Limited Adaptive Histogram Equalization," in Graphics Gems IV, P. S. Heckbert, Ed. San Diego, CA, USA: Academic Press, 1994, pp. 474–485.

[14] M. Z. Alom, T. M. Taha, C. Yakopcic, S. Westberg, P. Sidike, M. S. Nasrin, B. C. Van Emon, and V. K. Asari, "A State-of-the-Art Survey on Deep Learning Theory and Architectures," Electronics, vol. 8, no. 3, p. 292, Mar. 2019.

[15] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, 2016, pp. 770–778.

[16] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature Pyramid Networks for Object Detection," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), Honolulu, HI, USA, 2017, pp. 2117–2125.

[17] Y. Malkov and D. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," IEEE Trans. Pattern Analysis and Machine Intelligence, vol. 42, no. 4, pp. 824–836, Apr. 2020.

[18] R. Szeliski, Computer Vision: Algorithms and Applications, 2nd ed. Cham, Switzerland: Springer Nature, 2022.

[19] A. K. Jain, Fundamentals of Digital Image Processing. Englewood Cliffs, NJ, USA: Prentice Hall, 1989.

[20] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016.

[21] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," IEEE Trans. Pattern Analysis and Machine Intelligence, vol. 39, no. 6, pp. 1137–1149, June 2017.

[22] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal Loss for Dense Object Detection," in Proc. IEEE Int. Conf. Computer Vision (ICCV), Venice, Italy, 2017, pp. 2980–2988.

[23] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), New Orleans, LA, USA, 2022, pp. 11976–11986.

[24] B. Lubanovic, FastAPI: Modern Python Web Development. Sebastopol, CA, USA: O'Reilly Media, 2023.

[25] Celery Project, "Celery – Distributed Task Queue: Documentation," 2024. [Online]. Available: https://docs.celeryq.dev/

[26] World Wide Web Consortium (W3C), "Web Audio API," W3C Recommendation, June 2021. [Online]. Available: https://www.w3.org/TR/webaudio/

[27] Ionic Team, "Capacitor: Universal Cross-Platform Native App Runtime," Capacitor Documentation, 2024. [Online]. Available: https://capacitorjs.com/docs

[28] Ministry of Law and Justice (Government of India), "The Digital Personal Data Protection Act, 2023 (Act No. 22 of 2023)," The Gazette of India, Extraordinary, Part II, Section 1, Aug. 11, 2023.

[29] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 (General Data Protection Regulation)," Official Journal of the European Union, vol. L 119, pp. 1–88, May 2016.

[30] G. B. Huang, M. Mattar, T. Berg, and E. Learned-Miller, "Labeled Faces in the Wild: A Database for Studying Face Recognition in Unconstrained Environments," University of Massachusetts, Amherst, Tech. Rep. 07-49, Oct. 2007.

[31] S. Sengupta, J.-C. Chen, C. Castillo, V. M. Patel, R. Chellappa, and D. W. Jacobs, "Frontal to Profile Face Verification in the Wild," in Proc. IEEE Winter Conf. Applications of Computer Vision (WACV), Lake Placid, NY, USA, 2016, pp. 1–9.

[32] S. Moschoglou, A. Papaioannou, C. Sagonas, J. Deng, I. Kotsia, and S. Zafeiriou, "AgeDB: The First Manually Collected, In-the-Wild Age Database," in Proc. IEEE Conf. Computer Vision and Pattern Recognition Workshops (CVPRW), Honolulu, HI, USA, 2017, pp. 51–59.

[33] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning Transferable Visual Models From Natural Language Supervision," in Proc. 38th Int. Conf. Machine Learning (ICML), 2021, pp. 8748–8763.

[34] L. van der Maaten and G. Hinton, "Visualizing Data using t-SNE," Journal of Machine Learning Research, vol. 9, no. 86, pp. 2579–2605, 2008.

[35] J. Brooke, "SUS: A 'Quick and Dirty' Usability Scale," in Usability Evaluation in Industry, P. W. Jordan, B. Thomas, I. L. McClelland, and B. Weerdmeester, Eds. London, UK: Taylor & Francis, 1996, pp. 189–194.

[36] M. Abadi et al., "TensorFlow: A System for Large-Scale Machine Learning," in Proc. 12th USENIX Symp. Operating Systems Design and Implementation (OSDI), Savannah, GA, USA, 2016, pp. 265– 283.

[37] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in Advances in Neural Information Processing Systems (NeurIPS), Vancouver, BC, Canada, 2019, pp. 8024–8035.

[38] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, 2016, pp. 779–788.

[39] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, vol. 25, no. 11, pp. 120– 125, 2000.

[40] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image Quality Assessment: From Error Visibility to Structural Similarity," IEEE Trans. Image Processing, vol. 13, no. 4, pp. 600–612, Apr. 2004.

[41] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in Proc. 3rd Int. Conf. Learning Representations (ICLR), San Diego, CA, USA, 2015.

[42] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in Proc. 32nd Int. Conf. Machine Learning (ICML), Lille, France, 2015, pp. 448–456.

[43] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," Journal of Machine Learning Research, vol. 15, no. 56, pp. 1929–1958, 2014.

[44] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," in Advances in Neural Information Processing Systems (NeurIPS), Lake Tahoe, NV, USA, 2012, pp. 1097–1105.

[45] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention Is All You Need," in Advances in Neural Information Processing Systems (NeurIPS), Long Beach, CA, USA, 2017, pp. 5998–6008.

[46] R. Guo, P. Sun, E. Lindgren, Q. Geng, D. Simcha, F. Chern, and S. Kumar, "Accelerating Large-Scale Inference with Anisotropic Vector Quantization," in Proc. 37th Int. Conf. Machine Learning (ICML), 2020, pp. 3887–3896.

[47] B. McFee, C. Raffel, D. Liang, D. P. W. Ellis, M. McVicar, E. Battenberg, and O. Nieto, "librosa: Audio and Music Signal Analysis in Python," in Proc. 14th Python in Science Conf., Austin, TX, USA, 2015, pp. 18–25.

[48] Zulko et al., "MoviePy: Video Editing with Python," GitHub repository, 2024. [Online]. Available: https://github.com/Zulko/moviepy

[49] World Wide Web Consortium (W3C), "WebRTC: Real-Time Communication in Browsers," W3C Recommendation, Jan. 2021. [Online]. Available: https://www.w3.org/TR/webrtc/

[50] Vercel Inc., "Next.js Documentation," 2024. [Online]. Available: https://nextjs.org/docs

[51] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang, "Deep Learning with Differential Privacy," in Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS), Vienna, Austria, 2016, pp. 308–318.

[52] M. Kim, A. K. Jain, and X. Liu, "AdaFace: Quality Adaptive Margin for Face Recognition," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), New Orleans, LA, USA, 2022, pp. 18750–18759.

[53] D. P. W. Ellis, "Beat Tracking by Dynamic Programming," Journal of New Music Research, vol. 36, no. 1, pp. 51–60, 2007.

[54] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hays, F. Zhang, C.-L. Chang, M. G. Yong, J. Lee, W.-T. Chang, W. Hua, M. Georg, and M. Grundmann, "MediaPipe: A Framework for Building Perception Pipelines," arXiv preprint arXiv:1906.08172, 2019.

[55] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

APPENDIX A

APPENDIX A – SAMPLE CODE IMPLEMENTATION

This appendix reproduces short excerpts from the Memora source code. They illustrate the core implementations behind feature extraction, FAISS search, beat-synchronised reel generation and the fault-tolerant worker pipeline.

Listing A.1: SCRFD Multi-Scale Image Tiling & NMS Fusion (backend/services/tiling_service.py)

```
import cv2, numpy as np from typing import List, Dict  class
AdaptiveTilingDetector:     def __init__(self, detector, tile_size: int = 1920, overlap:
float = 0.20):
       self.detector = detector
       self.tile_size = tile_size
       self.step = int(tile_size * (1.0 - overlap))
    def detect_faces_tiled(self, img_bgr: np.ndarray) -> List[Dict]:
       H, W = img_bgr.shape[:2]
       all_boxes, all_kpss = [], []         if W
> 2560 or H > 1440:
          for y in range(0, H, self.step):
             for x in range(0, W, self.step):
                tile = img_bgr[y:min(y + self.tile_size, H), x:min(x + self.tile_size, W)]                  for f in
self.detector.get(tile):
                   f.bbox += np.array([x, y, x, y])
                   f.kps += np.array([x, y])
                   all_boxes.append(f.bbox)
                   all_kpss.append(f.kps)
       idx = cv2.dnn.NMSBoxes(all_boxes, [1.0] * len(all_boxes), 0.5, 0.45)
       return [all_boxes[i] for i in idx]
```

Listing A.2: ArcFace Embedding Extraction & L2 Normalization (backend/services/face_service.py)

```
import numpy as np, cv2 from insightface.app import
FaceAnalysis  class FaceRecognitionService:     def
__init__(self, model_name: str = 'buffalo_l'):
       self.app = FaceAnalysis(name=model_name,
                          providers=['CUDAExecutionProvider', 'CPUExecutionProvider'])
       self.app.prepare(ctx_id=0, det_size=(640, 640))
    def extract_normalized_embedding(self, face_bgr: np.ndarray) -> np.ndarray:
       lab = cv2.cvtColor(face_bgr, cv2.COLOR_BGR2LAB)
       lab[:, :, 0] = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8)).apply(lab[:, :, 0])
       faces = self.app.get(cv2.cvtColor(lab, cv2.COLOR_LAB2BGR))
if not faces:
          return None
       raw = faces[0].embedding.astype(np.float32)
       return raw / (np.linalg.norm(raw) + 1e-10)  # strict unit L2 norm
```

Listing A.3: FAISS Vector Similarity & Centroid Query (backend/services/vector_service.py)

```
import faiss, numpy as np from typing import
List, Tuple  class EventVectorIndex:     def
__init__(self, dimension: int = 512):
        self.index = faiss.IndexFlatIP(dimension)
        self.media_map: List[str] = []
    def add_vectors(self, vectors: np.ndarray, media_ids: List[str]):
        self.index.add(vectors.astype(np.float32))
        self.media_map.extend(media_ids)
    def query_matches(self, centroid: np.ndarray, tau: float = 0.65) -> List[Tuple[str, float]]:
        scores, idxs = self.index.search(centroid.reshape(1, -1).astype(np.float32),
                                         min(500, self.index.ntotal))
        return [(self.media_map[idx], float(sc))
                for sc, idx in zip(scores[0], idxs[0]) if sc >= tau]
```

Listing A.4: Beat-Synchronized Timeline Synthesis (frontend/src/lib/reelEditor.ts)

```
export function generateDynamicTimeline(mediaUrls: string[], bpm: number = 128) {   const beat = 60 / bpm;
  return mediaUrls.map((url, i) => ({
    id: `clip_${i}`,
    url,
    durationSec: (i % 3 === 0 ? 4 : 2) * beat,
    kenBurnsZoom: i % 3 === 0 ? { start: 1.0, end: 1.15 } : { start: 1.05, end: 1.0 }
  }));
}
```

Listing A.5: Celery Task Dispatcher with Thread Fallback (backend/api/media.py)

```
import threading  @router.post('/upload') async def
upload_event_media(event_id: str, file: UploadFile):     media =
save_media_metadata(event_id, file.filename)     try:
        return {'status': 'queued', 'task_id': process_media_task.delay(media.id).id}     except
Exception:
        threading.Thread(target=process_media_pipeline_sync,
                         args=(media.id,), daemon=True).start()
        return {'status': 'fallback_thread', 'media_id': media.id}
```

Listing A.6: Procedural Web Audio API Synthesizer (frontend/src/lib/reelAudio.ts)

```
export class ProceduralSynthesizer {   private ctx = new (window.AudioContext || (window as any).webkitAudioContext)();

  playSynthNote(freq: number, startTime: number, duration: number) {
    const osc = this.ctx.createOscillator();
    const gain = this.ctx.createGain();
    const filter = this.ctx.createBiquadFilter();
    osc.type = 'sawtooth';
    osc.frequency.setValueAtTime(freq, startTime);
```

```
    filter.type = 'lowpass';
    filter.frequency.setValueAtTime(2000, startTime);
    gain.gain.setValueCurveAtTime([0, 0.3, 0.1, 0], startTime, duration);
    osc.connect(filter).connect(gain).connect(this.ctx.destination);
    osc.start(startTime);
    osc.stop(startTime + duration);
  }
}
```

Listing A.7: Capacitor Android Cleartext & Permissions Config (frontend/capacitor.config.ts)

```
import { CapacitorConfig } from '@capacitor/cli';
const config: CapacitorConfig = {   appId:
'com.memora.app',
  appName: 'Memora',
  webDir: 'out',
  server: { url: 'http://192.168.1.10:8000', cleartext: true }, // local Wi-Fi binding for Oppo
Reno 11
  android: { allowMixedContent: true }
};  export default
config;
```